

《现代C++编程实现》电子版: <https://leanpub.com/c01>

第6章 函数

YouTube频道: [hwdong](https://www.youtube.com/hwdong)

博客: hwdong-net.github.io

6.1函数是命名的程序块

- **6.1.1 最大公约数**
- **6.1.2 函数的定义**
- **6.1.3 函数返回值和void函数**
- **6.1.4 标准库函数**

6.1.1 最大公约数

- 两个正整数的最大公约数就是能被两者整除的最大正整数。

$$GCD(m, n) = \begin{cases} m & n=0 \\ GCD(n, m\%n) & n \neq 0 \end{cases}$$

$$\begin{aligned} GCD(72, 27) &= GCD(27, 72\%27) = GCD(27, 18) \\ &= GCD(18, 27\%18) = GCD(18, 9) \\ &= GCD(9, 0) = 9 \end{aligned}$$

6.1.1 最大公约数

- 两个正整数的最大公约数就是能被两者整除的最大正整数。

```
#include <iostream>
```

```
int main(){
```

```
    int m = 72,n = 27;
```

```
    while(n!=0){
```

```
        int r = m%n;
```

```
        m = n; n = r;
```

```
    }
```

```
    std::cout<<"最大公约数是:  "<<m<<std::endl;
```

```
}
```

$$GCD(m,n) = \begin{cases} m & n=0 \\ GCD(n,m\%n) & n!=0 \end{cases}$$

- 又要求另外2个整数（比如36 和24）的最大公约数，怎么办？

复制、黏贴、修改代码

```
#include <iostream>
```

```
int main(){
```

```
    int m = 72,n = 27;
```

```
    while(n!=0){
```

```
        int r = m%n;
```

```
        m = n; n = r;
```

```
    }
```

```
    std::cout<<m<<"和"<<n<<"的最大公约数是： "<<m<<std::endl;
```

```
    m = 36; n = 24;
```

```
    while(n!=0){
```

```
        int r = m%n;
```

```
        m = n; n = r;
```

```
    }
```

```
    std::cout<<m<<"和"<<n<<"的最大公约数是： "<<m<<std::endl;
```

```
}
```

6.1.2 函数的定义与调用

- 给“求最大公约数的这段代码”起一个名字，即定义所谓的**函数**

```
// 函数定义: GCD 是函数名, m 和 n 是形式参数
// 函数名前的 void 表示函数没有返回值
void GCD(int m, int n) {
    while (n != 0) {
        int r = m % n;
        m = n;
        n = r;
    }
    std::cout << "最大公约数是:  " << m << std::endl;
}
```

6.1.2 函数的定义与调用

- 通过**函数名**可以调用执行函数中的代码，调用函数时将**实际参数**（简称**实参**）传递给**被调用函数**的**形参**

```
int main() {  
    int x = 72, y = 27;  
    GCD(x, y); // 调用函数 GCD, 将 x 和 y 的值传递给 m 和 n  
  
    x = 36;  
    y = 24;  
    GCD(x, y);  
}
```

一次编写、多次调用

一次编写、多次调用

6.1.3 返回值与 void 函数

- 函数名前的 void 表示的是函数的返回值类型，返回值类型是void的叫void 函数，表示不返回值。

```
// 函数定义: GCD 是函数名, m 和 n 是形式参数
// 函数名前的 void 表示函数没有返回值
void GCD(int m, int n) {
    while (n != 0) {
        int r = m % n;
        m = n;
        n = r;
    }
    std::cout << "最大公约数是: " << m << std::endl;
}
```

6.1.3 返回值与 void 函数

- 函数也可以有返回值，例如，将 GCD 函数修改为返回最大公约数。

```
// 函数名前面的 int 表示函数返回 int 类型的值
int GCD(int m, int n) {
    while (n != 0) {
        int r = m % n;
        m = n;
        n = r;
    }
    return m; // 返回 m 的值
}
```

- 在 main 中调用并接收返回值

```
int main() {  
    int x = 72, y = 27;  
    int gcd = GCD(x, y); // 用函数返回的结果初始化变量 gcd  
    std::cout << x << " 和 " << y << " 的最大公约数是: " << gcd << std::endl;  
  
    x = 36;  
    y = 24;  
    gcd = GCD(x, y);  
    std::cout << x << " 和 " << y << " 的最大公约数是: " << gcd << std::endl;  
}
```

6.1.4 标准库函数举例

- 数学函数库<cmath>提供了许多数学计算的函数。
- 如求平方根的 sqrt() 函数、指数函数 pow() 等

```
#include <cmath>    // 包含声明 sqrt 和 pow 函数的头文件 cmath
#include <iostream>

int main() {
    double d = 6.8;
    std::cout << d << " 的平方根是: " << sqrt(d) << std::endl;
    std::cout << d << " 的 3.4 次方是: " << pow(d, 3.4) << std::endl;
}
```

6.2 函数的定义与声明

在 C++ 中，函数的声明和定义是两个不同的概念。

- **函数声明**：告诉编译器**函数的名称、返回类型以及参数类型**，但不包含函数体。
- **函数定义**：则包括函数的完整实现，包括函数体。

6.2.1 函数声明

返回类型 函数名(参数类型列表);

例如:

```
int GCD(int m, int n);
```

该声明表示一个名为 GCD 的函数，接受两个 int 类型的参数，并返回一个 int 类型的结果。

- 在函数声明中，只需提供函数返回类型、函数名以及形参的类型和顺序，形参名是可选的

```
int GCD(int, int); // 仅提供参数类型，无需提供参数名
```

这在实际开发中尤其常见于头文件中，仅仅用来暴露函数接口，而将实现细节留在源文件中。

6.2.2 函数的定义

- 函数定义 (Function Definition) 包含函数的完整实现，包括函数体。

```
返回类型 函数名(参数类型列表) {  
    函数体;  
}
```

6.2.2 函数的定义

- 函数定义 (Function Definition) 包含函数的完整实现，包括函数体。

```
int GCD(int m, int n) {  
    while(n != 0) {  
        int r = m % n;  
        m = n;  
        n = r;  
    }  
    return m;  
}
```

6.2.2 函数的定义

- 注意： 函数的定义中包含了函数体，因此不需要以分号结尾。

```
int GCD(int m, int n) {  
    while(n != 0) {  
        int r = m % n;  
        m = n;  
        n = r;  
    }  
    return m;  
}
```

6.2.3 函数签名 (Function Signature)

函数签名由以下部分组成：

- 函数名
- 参数类型列表（包括参数类型和顺序）

例如，以下两个函数具有不同的签名：

```
int add(int x, int y);    // 签名为 add(int, int)
double add(double x, double y); // 签名为 add(double, double)
```

6.2.3 函数签名 (Function Signature)

注意：

- 在C++中可以定义同名但不同签名的函数，称为函数重载。
- 函数签名不包括返回类型，这意味着不能仅通过返回类型不同来区分两个函数。
- 函数签名在函数重载中起到关键作用，在后续章节（函数重载）中会详细讨论。

```
int add(int x, int y);      // 签名为 add(int, int)
double add(double x, double y); // 签名为 add(double, double)
```

6.2.4 函数声明与定义的关系

函数声明和定义可以分开，也可以合并。

- **分开形式：** 先声明，再定义。例如：

```
// 函数声明
int GCD(int m, int n);

int main() {
    int result = GCD(72, 27);
    std::cout << "最大公约数是: " << result << std::endl;
}
```

在这种形式中，函数声明通常放在 main() 函数之前，函数定义可放在 main() 函数之后。

6.2.4 函数声明与定义的关系

函数声明和定义可以分开，也可以合并。

- **分开形式：** 先声明，再定义。例如：

```
// 函数定义
int GCD(int m, int n) {
    while (n != 0) {
        int r = m % n;
        m = n;
        n = r;
    }
    return m;
}
```

6.2.4 函数声明与定义的关系

- **合并形式：** 直接在声明处完成定义，例如：

```
int GCD(int m, int n) {  
    while (n != 0) {  
        int r = m % n;  
        m = n;  
        n = r;  
    }  
    return m;  
}
```

6.2.5 函数声明与定义分开的实际应用：一个简单的工程项目示例

- 在一个复杂的工程中，代码通常被分成多个源文件（.cpp 文件）和头文件（.h 或 .hpp 文件）。在头文件中进行函数的声明，在源文件中定义函数。
- 这种分离的结构有助于代码的模块化和重用。而函数声明与定义的分开，便于在不同的源文件之间进行调用。

示例工程结构：

```
calculator_project/  
|  
├─ main.cpp           // 主程序文件  
├─ math_functions.cpp // 数学函数实现文件  
├─ math_functions.h   // 数学函数声明文件  
└─ CMakeLists.txt     // 构建脚本
```

math_functions.h (头文件, 声明函数)

```
// math_functions.h
#ifndef MATH_FUNCTIONS_H
#define MATH_FUNCTIONS_H

// 函数声明
int GCD(int m, int n);
int LCM(int m, int n);

#endif // MATH_FUNCTIONS_H
```

- 头文件 math_functions.h 中只包含函数的声明。
- 使用 #ifndef 、#define 和 #endif 可以防止头文件的内容被重复包含, 这称为 **头文件保护**, 确保其内容在同一编译单元中只被编译一次。

math_functions.cpp (源文件, 实现函数)

```
// math_functions.cpp
#include "math_functions.h"
#include <iostream>

int GCD(int m, int n) {
    while (n != 0) {
        int r = m % n;
        m = n;
        n = r;
    }
    return m;
}

int LCM(int m, int n) {
    return (m * n) / GCD(m, n);
}
```

math_functions.cpp 通常包含函数的具体实现。

通过 `#include "math_functions.h"` , 可以在该源文件中使用对应的函数声明。

main.cpp (主程序文件, 调用函数)

```
// main.cpp
#include <iostream>
#include "math_functions.h" // 引入函数声明

int main() {
    int x = 72, y = 27;
    std::cout << "最大公约数是: " << GCD(x, y) << std::endl;
    std::cout << "最小公倍数是: " << LCM(x, y) << std::endl;
    return 0;
}
```

main.cpp 通过 `#include "math_functions.h"` 使得 GCD 和 LCM 的声明可用, 从而可以在 `main()` 中调用这些函数。

main.cpp (主程序文件, 调用函数)

```
// main.cpp
#include <iostream>
#include "math_functions.h" // 引入函数声明

int main() {
    int x = 72, y = 27;
    std::cout << "最大公约数是: " << GCD(x, y) << std::endl;
    std::cout << "最小公倍数是: " << LCM(x, y) << std::endl;
    return 0;
}
```

注意: 在 main.cpp 中, 只需包含头文件, 而不能直接包含 math_functions.cpp, 因为源文件应由编译器分别编译后在链接阶段合并。

编译与链接

- 编译与链接当你将这些文件放在一起时，你可以使用 编译器（如 g++ ）来编译并链接这些文件，形成最终的可执行文件：

```
g++ main.cpp math_functions.cpp -o calculator
```

为什么要分开声明与定义？

- **ODR (One Definition Rule, 唯一定义规则)**。该规则规定：在整个程序中，对于非内联函数、全局变量等实体，只能有一个定义。
- 如果函数定义被放在头文件中，且该头文件被多个源文件包含，就会导致重复定义，引发链接错误。

分开声明与定义的好处

- 1. **代码模块化**：将函数接口放在头文件中，实现细节隐藏于源文件中，对外仅暴露声明，有助于封装。
- 2. **避免重复定义**：声明与定义分开，保证了整个程序只有一份函数实现，符合 ODR 要求。
- 3. **便于重用与维护**：修改函数实现只需改动源文件，头文件不变，其他调用该函数的文件无需重新修改，提高了可维护性。
- 4. **提高编译效率**：修改一个源文件只需重新编译该文件，而无需重新编译所有文件，加快大型项目的构建速度。

6.3 函数参数

- 6.3.1 参数传递
- 6.3.2 默认参数
- 6.3.3 数组作为形参
- 6.3.4 `const` 与形参
- 6.3.5 可变数目的形参

6.3.1 参数传递

调用函数时，形参被创建并用实参初始化。C++ 中参数的传递方式分为两种：

- **值传递：**将实参的值拷贝给形参，形参和实参占用不同的内存空间，函数内对形参的修改不会影响实参。包括普通变量和指针变量的传递。
- **引用传递：**形参是实参的别名，形参和实参共享同一块内存，函数内对形参的修改会直接影响实参。

1. 值传递

- 在值传递中，形参被初始化为实参的拷贝，函数内对形参的修改不会影响实参。

```
#include <iostream>
void increment(int x) {
    x++;
    std::cout << "Inside function: x = " << x << std::endl;
}

int main() {
    int num = 5;
    increment(num);
    std::cout << "After function call: num = " << num << std::endl;
}
```

示例：指针的值传递

- 指针作为实参传递时，本质上也是值传递，即传递的是指针变量的值，但指针指向的内存可以被修改。

```
#include <iostream>

void changeValue(int* ptr) {
    *ptr = 10;
}

int main() {
    int num = 5;
    changeValue(&num);
    std::cout << "After function call: num = " << num << std::endl;
}
```

- 虽然 ptr 是指针的值拷贝（即地址的拷贝），但它指向的内存地址与实参相同，因此可以通过该地址间接修改指针所指向的变量 num 的值。

```
#include <iostream>

void changeValue(int* ptr) {
    *ptr = 10;
}

int main() {
    int num = 5;
    changeValue(&num);
    std::cout << "After function call: num = " << num << std::endl;
}
```


2. 引用传递

- 在引用传递中，形参是实参的别名，它们共享同一块内存，因此对形参的修改就是对实参的修改。

```
#include <iostream>
void increment(int& x) {
    x++;
}

int main() {
    int num = 5;
    increment(num);
    std::cout << "After function call: num = " << num << std::endl;
}
```

6.3.2 默认参数

- C++ 允许函数形参指定默认值，调用函数时若省略对应的实参，则会自动使用默认值。

```
#include <iostream>

int power(int base, int exp = 2) {
    int result = 1;
    for (int i = 0; i < exp; ++i) {
        result *= base;
    }
    return result;
}

int main() {
    std::cout << power(3) << std::endl;           // exp使用默认值 2，输出 9
    std::cout << power(3, 3) << std::endl;         // exp使用传入值 3，输出 27
}
```

- 默认参数必须从右向左连续出现，即有默认值的参数必须位于无默认值的参数之后。
- 如果调用时传递了对应实参，则会覆盖默认值。

```
#include <iostream>

int power(int base, int exp = 2) {
    int result = 1;
    for (int i = 0; i < exp; ++i) {
        result *= base;
    }
    return result;
}

int main() {
    std::cout << power(3) << std::endl;           // exp使用默认值 2，输出 9
    std::cout << power(3, 3) << std::endl;         // exp使用传入值 3，输出 27
}
```

6.3.3 数组作为形参

- 在 C++ 中，函数的形参可以写成数组的形式，但实际上，这个数组形参并不是真正的数组，而是一个指向数组的指针变量。编译器会将数组形参自动转换为指针形式。

```
void PrintArr(int arr[], int n) {  
    for (int i = 0; i < n; i++) {  
        std::cout << arr[i] << '\t';  
    }  
}
```

```
void PrintArr(int arr[], int n) {  
    for (int i = 0; i < n; i++) {  
        std::cout << arr[i] << '\t';  
    }  
}
```

- 编译器实际上将其转换为如下形式：

```
void PrintArr(int *arr, int n);
```

- 形参 arr 实际上是一个 int* 类型的指针，而不是一个真正的数组。因此，不能对 arr 使用 Range for 循环去访问数组元素。

```
void PrintArr(int arr[], int n) {  
    for (auto e : arr) // 错误, arr 并不是一个真正的数组  
        std::cout << e << '\t';  
}
```

将产生编译错误：

```
error: range-based 'for' loop requires a suitable begin function
```

- 在数组形参中的 [] 里指定数组大小是没有任何意义的。以下几种形式都是等价的，并且最终都会被编译器转换为指针形式：

```
void PrintArr(int arr[], int n);  
void PrintArr(int arr[2], int n);  
void PrintArr(int arr[10], int n);  
void PrintArr(int *arr, int n); // 最终都会转化为这个形式
```

- 由于 arr 实际上是一个指针变量，因此可以通过指针遍历数组中的元素。

```
void PrintArr(int arr[], int n) {  
    for (auto p = arr; p != arr + n; p++)  
        std::cout << *p << '\t';  
}
```


引用数组形参：

- 如果形参是数组引用，则引用的是整个实参数组，编译器不会将其转换为指针。

```
#include <iostream>
void squareArray(int (&arr)[5]) {
    for (int &num : arr) {
        num *= num;
    }
}

int main() {
    int nums[] = {1, 2, 3, 4, 5};
    squareArray(nums);
    for (int num : nums) {
        std::cout << num << " ";
    }
}
```

引用数组形参：

- **注意：** 引用数组形参要求数组大小必须明确，且实参和形参的大小必须一致。

```
#include <iostream>
void squareArray(int (&arr)[5]) {
    for (int &num : arr) {
        num *= num;
    }
}

int main() {
    int nums[] = {1, 2, 3, 4, 5};
    squareArray(nums);
    for (int num : nums) {
        std::cout << num << " ";
    }
}
```

6.3.4 const 与形参

- 当 const 与函数形参结合时，既可以修饰参数本身，使其在函数内部不可修改，也可以修饰指针指向的对象，使该对象不可修改。
例如：

```
void f(const int x);  
void g(const int *p);  
void h(int *const q);  
void k(const int *const s);
```

f()

- 形参 x 是 const int 类型，这意味着在 f() 函数内部，它是不可修改的 const int 变量。例如，以下操作会产生编译错误：

```
void f(const int x) {  
    x = 10;  // 错误：不能修改 const int 变量  
}
```

g()

- 形参 p 是 `const int*` 类型，即指向 `const int` 的指针。这意味着在 `g()` 函数内部，不能通过 p 修改它所指向的 `const int` 变量，但 p 本身可以被修改，例如：

```
void g(const int *p) {  
    *p = 10;    // 错误：不能修改 const int 类型的值  
    p++;        // 合法：可以修改指针本身  
}
```

h()

- 形参 q 是 `int* const` 类型，即 q 本身是 `const` 指针，但它指向的 `int` 类型变量可以被修改。
- 这意味着在 `h()` 函数内部，q 本身不能被修改，但可以通过 q 修改它所指向的 `int` 变量，例如：

```
void h(int *const q) {  
    q = nullptr;    // 错误：不能修改 const 指针  
    *q = 10;        // 合法：可以修改指针指向的变量  
}
```

指向 const 的 const 指针

- 如果既不允许修改指针本身，也不允许修改指针所指向的变量，则可以将指针定义为 const 指针，且指向 const 对象，例如：

```
void k(const int *const s) {  
    // s 本身是 const，不能修改指向，*s 也是 const，不能修改  
    std::cout << *s << std::endl;  
    *s = 10; // 错误：不能修改 s指向的const int对象  
    s++;    // 错误：不能修改 const 指针  
}
```

数组形参与 const 的结合

- 数组形参实际上是指向数组的指针，因此数组形参与 const 的结合与指针类似。例如：

```
void PrintArr(const int arr[], int n);
```

等价于：

```
void PrintArr(const int *arr, int n);
```

在 PrintArr() 函数中，不允许修改 arr 指向的数组内容，但可以修改 arr 本身，如进行指针运算。

引用形参与 const 的结合

- 与指针类似，引用形参也可以被 const 修饰，以防止修改被引用的对象。例如：

```
void Print(const std::string &str);
```

在函数内部，无法通过 str 修改所引用的字符串内容。

const 引用不仅避免不必要的拷贝，提高效率，还能确保函数不会修改传入的对象。通常用于传递较大的对象，但在某些情况下也适用于小型对象。

6.3.5 可变数目的形参

在编程中，有时无法提前预知传递给函数的参数个数，例如：

- 计算学生的平均分，但课程门数未知。
- 打印日志时，输出内容的数量和类型不固定。
- 虽然可以通过 **数组指针和数组大小** 来解决，但如果希望 **直接传递一组数值** 而不额外传递数量，则可以使用 **可变数目的形参**。

使用 `std::initializer_list<T>` 实现可变参数

C++ 提供了 `std::initializer_list<T>`，用于定义 **数量可变** 且 **类型固定** 的参数。

- `initializer_list` 是 **类模板**（第 11 章会介绍类模板），需要通过 `<T>` 指定具体的数据类型。
- 它能够 **接收并存储** 传递的一组实参，并在函数中通过 **Range-for** 循环 进行访问。
- 其中的元素 **类型相同**，且在函数内部 **不可修改**（存储的值是 `const`）。

语法形式：

```
返回类型 函数名(std::initializer_list<T> 参数名);
```

```
#include <iostream>
#include <initializer_list>

// 定义函数，使用 std::initializer_list<double> 形参
double average(std::initializer_list<double> scores) {
    double all{ 0 }; // 总分

    // 使用 Range for 循环遍历 initializer_list
    for (auto score : scores) {
        all += score;
    }

    // 使用 size() 函数获取分数个数
    auto n = scores.size();

    return (n > 0) ? all / n : 0; // 计算平均分
}
```

```
double average(std::initializer_list<double> scores) {  
    double all{ 0 }; // 总分  
  
    // 使用 Range for 循环遍历 initializer_list  
    for (auto score : scores) {  
        all += score;  
    }  
    // 使用 size() 函数获取分数个数  
    auto n = scores.size();  
    return (n > 0) ? all / n : 0; // 计算平均分  
}
```

```
int main() {  
    std::cout << average({}) << '\n'; // 没有参数, 输出: 0  
    std::cout << average({60.0}) << '\n'; // 一门课程, 输出: 60  
    std::cout << average({50.0, 80.0}) << '\n'; // 两门课程, 输出: 65  
    std::cout << average({90.0, 50.0, 80.0}) << '\n'; // 三门课程, 输出: 73.3333  
    return 0;  
}
```

注意事项

1. `std::initializer_list<T>` 中的元素是 只读的，在函数内部 不可以修改。
2. 传递给 `std::initializer_list<T>` 的参数必须是同一种类型 `T`，否则会出现编译错误，例如：

```
average({90, 85.5, "Hello"}); // 错误：类型不一致
```

3. 要使用 `std::initializer_list`，需要包含头文件 `<initializer_list>`，但 `<iostream>` 已经包含了这个头文件，因此无需额外导入。

6.4 返回值

函数可以将计算结果返回给调用者，也可以只执行操作而不返回任何值。函数定义和声明中都必须明确指定函数的返回值类型，返回值类型可以是基本类型、指针、引用、自定义类型或 `void`。

6.4 返回值

- 6.4.1 返回值类型
- 6.4.2 **return 语句**
- 6.4.3 返回值的类型转换
- 6.4.4 返回引用
- 6.4.5 返回指针
- 6.4.6 返回 const 值、引用和指针
- 6.4.7 返回值优化

6.4.1 返回值类型

- 函数的返回值类型在 函数声明和定义 中必须 明确指定，并且只能有一个返回值类型。返回值类型可以是：
 - 基本数据类型（如 `int`、`double`、`char` 等）
 - 指针类型（如 `int*`、`double*` 等）
 - 引用类型（如 `int&`、`double&` 等）
 - 自定义类型（如类、结构体、枚举等）
 - `void` 类型（表示 无返回值）

6.4.1 返回值类型

- 函数的返回值类型在 函数声明和定义 中必须 明确指定，并且只能有一个返回值类型。返回值类型可以是：

例如：

```
int Add(int a, int b);           // 返回 int 类型
double Max(double x, double y); // 返回 double 类型
void PrintMessage();             // 无返回值，返回类型为 void
```

6.4.2 return 语句

- 函数通过 **return 语句** 返回值，语法为：

```
return 表达式;
```

- 执行 `return` 后，函数立即结束，并将表达式的值返回给调用者。
- 如果函数返回类型为 `void`，可以省略 `return` 或写为 `return;`（不带表达式）。

```
int add(int a, int b) {  
    return a + b;           // 返回计算结果  
}  
  
void printSum(int a, int b) {  
    std::cout << a + b;     // 无返回值，无需 return  
}
```

6.4.2 返回值的类型转换

- 当返回值类型与 return 后的表达式类型不同时，编译器会尝试进行类型转换。

```
int GetNumber() {  
    double x = 3.14;  
    return x;    // 返回类型为 int, x 被转换为 3  
}
```

注意：这种转换可能会 丢失精度 或 改变数据，需要 小心处理。

6.4.3 返回引用

- C++ 函数可以返回引用，其语法为：

```
类型 & 函数名(参数列表) {  
    // 函数体  
    return 引用;  
}
```

返回引用的主要作用是提高性能和允许修改被引用的对象。

返回引用的 主要作用 是 提高性能 和 允许修改被引用的对象。

```
int numbers[] = {1, 2, 3, 4};

int& GetElement(int arr[], int index) {
    return arr[index];
}

int main() {
    GetElement(numbers, 2) = 100;    // 修改 numbers[2] 的值
    std::cout << numbers[2];        // 输出 100
    return 0;
}
```

重要警告：永远不要返回局部变量的引用！

```
int& badReturn() {  
    int localVar = 10;  
    return localVar; // 返回局部变量的引用，局部变量会在函数结束时销毁  
}
```

局部变量在函数返回后即被销毁，返回其引用会导致未定义行为。

```
int main() {  
    int& ref = badReturn(); // 获取返回的引用  
    std::cout << ref << std::endl; // 未定义行为，可能会输出垃圾值  
    return 0;  
}
```

```
#include <iostream>

// 错误的返回局部变量的引用
int& ErrorReturn() {
    int localVar = 10;
    return localVar; // 返回局部变量的引用，局部变量会在函数结束时销毁
}

int main() {
    int& ref = ErrorReturn(); // 获取返回的引用
    std::cout << ref << std::endl; // 未定义行为，可能会输出垃圾值
    return 0;
}
```


6.4.4 返回指针

- 函数也可以 返回指针，其语法为：

```
类型* 函数名(参数列表) {  
    // 函数体  
    return 指针;  
}
```

返回指针的 主要作用 是 返回动态分配的内存 或 引用函数外部的变量。

例如：

```
int* AllocateMemory() {  
    int* p = new int(42);    // 动态分配内存  
    return p;  
}  
  
int main() {  
    int* ptr = AllocateMemory();  
    std::cout << *ptr << '\n';    // 输出 42  
    delete ptr;                    // 记得释放内存  
    return 0;  
}
```

同样警告：不要返回局部变量的地址！

```
int* badReturn() {  
    int local = 10;  
    return &local;    // 错误：返回局部变量的地址  
}  
  
int main() {  
    int* p = ErrorPtr();    // 获取返回的指针  
    std::cout << *p << std::endl;    // 未定义行为，可能会输出垃圾值  
    return 0;  
}
```

注意:

- 返回指针时，不要返回局部变量的地址，因为局部变量在函数结束后会被销毁。
- 返回 动态分配的内存 时，调用者需要 手动释放内存，以防内存泄漏。

错误示例:

```
int* ErrorPtr() {  
    int x = 10;  
    return &x;    // 错误: 返回局部变量的地址  
}
```

6.4.5 返回 const 值、const引用和const指针

- 返回 `const` 值：用于 防止调用者修改返回值，适用于类对象的只读访问。
- 返回 `const` 引用：用于 提高效率 并 防止调用者修改引用的对象。
- 返回 `const` 指针：用于 防止调用者通过指针修改指向的对象。

1. 返回 const 值

```
const int GetConstValue() {  
    return 42;  
}
```

返回普通值时加 `const` 通常没有实际作用，因为返回的是临时副本，调用者无法通过它影响原变量。因此这种用法很少见，一般省略 `const`。

2. 返回 const对象的 引用

```
const int& GetConstRef(const int& x) {  
    return x;  
}
```

作用：

- 避免拷贝，提高性能（尤其对大对象）。
- 防止调用者通过返回值修改对象。
- 常用于返回容器元素或类成员，提供只读访问。

正确用法：

```
const std::string& getData(const std::string& data) {  
    // 处理数据...  
    return data;    // 返回 const 引用，只读访问  
}
```

错误示例（返回局部变量的 const 引用）：

```
const std::string& badReturn() {  
    std::string local = "Hello";  
    return local;    // 错误：局部变量被销毁，返回悬垂引用  
}
```


3. 返回 const对象的 指针

```
const int* GetConstPointer(const int* p) {  
    return p;  
}
```

调用者不能通过返回的指针修改指向的对象，
但指针本身可重新指向。

4. 返回 const 指针

返回的是 const 指针，意味着指针本身不能被修改，但指向的对象（变量）可以被修改。

```
int* const getConstPointer(int* p) {  
    return p;           // 返回 const 指针，指针本身不可修改  
}
```

5. 返回指向 const 对象的 const 指针

```
const int* const getConstPtrToConst(const int* p) {  
    return p;          // 指针本身和指向的对象都不可修改  
}
```

6.4.6 返回值优化

C++ 编译器在返回对象时会进行**返回值优化**，避免不必要的拷贝，提高性能。

```
std::string getString() {  
    std::string s = "Hello";  
    return s;           // RVO 可能直接在调用者的内存中构造 s  
}
```

- **RVO（返回值优化）**：编译器直接在调用者的内存空间构造返回值，消除临时对象和拷贝。
- **移动语义**：如果无法应用 RVO，编译器会尝试调用移动构造函数转移资源，而非深拷贝。

```
std::string getString() {  
    std::string s = "Hello";  
    return s;           // RVO 可能直接在调用者的内存中构造 s  
}
```

没有 RVO 时的执行流程：

1. 在函数内创建局部对象 `s`。
2. 将 `s` 拷贝或移动到临时对象。
3. 销毁局部对象 `s`。
4. 将临时对象拷贝或移动到调用者变量。

```
std::string getString() {  
    std::string s = "Hello";  
    return s;           // RVO 可能直接在调用者的内存中构造 s  
}
```

有 RVO 时的优化:

- 编译器直接将 `s` 构造在调用者变量的内存位置上, 省略所有中间步骤。

6.4.7 小结

返回值形式	调用者能否修改	适用场景
普通值返回	不能（操作的是副本）	小型对象、基本类型
引用返回	能（若返回非 const 引用）	允许修改原对象、避免拷贝
const 引用返回	不能	只读访问、避免拷贝
指针返回	能（通过指针修改）	动态内存、可选空状态
const 指针返回	不能修改指针本身，但可修改指向对象	固定指针、可变内容
指向 const 的 const 指针	都不能修改	完全只读

核心原则：

- **不要返回局部变量的引用或指针**，否则导致悬垂引用/指针，引发未定义行为。
- 返回引用或指针时，确保被引用/指向的对象在函数返回后依然有效。
- 合理使用 `const` 可以提供只读访问，增强代码安全性。
- 现代编译器会进行返回值优化，编写代码时不必过度担心拷贝开销。

6.5 静态变量

- 在 C++ 中，变量的存储方式和作用域（可见范围）是影响程序行为的重要因素。除了普通的局部变量和全局变量，C++ 还提供了**静态变量（static 变量）**，它具有特殊的存储方式和作用域。
- 在 C++ 中，静态变量指的是具有静态存储周期（static storage duration）的变量，即它们在**程序运行期间始终存在，不会随作用域结束而销毁**。

6.5 静态变量

- 静态变量主要包括 **静态局部变量**（局部作用域，但在整个程序运行期间存在）、**文件作用域的静态变量**（定义在文件范围内，具有内部链接）以及 **类的静态成员变量**（属于类，而非具体对象）。
- 本章主要介绍 **静态局部变量** 和 **文件作用域的静态变量**，并通过示例来帮助理解它们的特点和作用。

6.5.1 静态局部变量

- 静态局部变量是在程序块内部使用 **static 关键字** 修饰的变量。

```
void func() {  
    static int count = 0; // 静态局部变量  
    count++;  
    std::cout << "count = " << count << std::endl;  
}
```

6.5.1 静态局部变量

- 静态局部变量的作用域也仅限于定义它的程序块内，但与普通局部变量不同的是：静态局部变量的生命周期贯穿整个程序的运行过程，即使程序块执行结束，变量也不会被销毁，而是继续保持其值，直到程序结束。

```
void func() {  
    static int count = 0; // 静态局部变量  
    count++;  
    std::cout << "count = " << count << std::endl;  
}
```

在不同程序块中的静态局部变量

- 静态局部变量不仅可以定义在函数中，也可以定义在 if 语句块、循环体等任何程序块内：

```
int main() {  
    if (true) {  
        static int x = 10; // 仅在 if 语句块内可见  
        std::cout << "Inside if block, x = " << x << std::endl;  
        x++;  
    }  
  
    for (int i = 0; i < 3; i++) {  
        static int y = 5; // 仅在 for 语句块内可见  
        std::cout << "Inside for loop, y = " << y << std::endl;  
        y++;  
    }  
  
    return 0;  
}
```

静态局部变量的初始化

- 静态局部变量在程序执行到该变量的定义时初始化，并且仅初始化一次。
- 如果没有显式赋值，静态局部变量的默认值是 0（与全局变量相同）。

```
void func() {  
    static int x; // 没有初始化，默认值为 0  
    x++;  
    std::cout << "x = " << x << std::endl;  
}  
  
int main() {  
    func(); // 输出 x = 1  
    func(); // 输出 x = 2  
    func(); // 输出 x = 3  
    return 0;  
}
```

返回静态局部变量的引用或指针

- 在 C++ 中，不能返回**非静态局部变量**的引用或指针，因为它们在函数返回后会被销毁，导致悬垂引用或指针。然而，
- **静态局部变量**的**生命周期贯穿整个程序运行**，因此可以安全地返回它的引用或指针，以在函数调用间保持数据。

```
#include <iostream>
```

```
int& getCounter() {
```

```
    static int counter = 0; // 静态局部变量，仅初始化一次
```

```
    return counter; // 返回引用，使得外部可以修改它
```

```
}
```

```
int main() {
```

```
    getCounter() += 1;
```

```
    std::cout << "Counter: " << getCounter() << std::endl; // 输出 1
```

```
    getCounter() += 1;
```

```
    std::cout << "Counter: " << getCounter() << std::endl; // 输出 2
```

```
}
```

它的存储在多次调用之间保持不变，但值可以被修改。
类似于全局变量，但其作用域受限于 getCounter() 函数。

它的存储在多次调用之间保持不变，但值可以被修改。
类似于全局变量，但其作用域受限于 getCount() 函数。

```
#include <iostream>

int* getCount() {
    static int counter = 0; // 静态局部变量，仅初始化一次
    return &counter; // 返回指针，使得外部可以修改它
}

int main() {
    *getCounter() += 1;
    std::cout << "Counter: " << *getCounter() << std::endl; // 输出 1

    *getCounter() += 1;
    std::cout << "Counter: " << *getCounter() << std::endl; // 输出 2
}
```

此外，可以提供只读版本：

```
const int& getConstCounter() {  
    static int counter = 0;  
    return counter; // 只读访问，外部无法修改  
}
```

静态局部变量的应用

1. 计数器（记录函数调用次数）

```
void countCalls() {  
    static int count = 0;  
    count++;  
    std::cout << "Function called " << count << " times.\n";  
}
```

静态局部变量的应用

2. 缓存数据（提高效率，查表法）

```
int expensiveComputation(int index) {  
    static int cache[10] = {-1, -1, -1, -1, -1, -1, -1, -1, -1, -1}; // 初始化为 -1  
    if (cache[index] != -1) return cache[index]; // 如果已有结果，直接返回  
    cache[index] = index * index; // 计算并存储  
    return cache[index];  
}  
  
int main() {  
    std::cout << expensiveComputation(3) << std::endl; // 计算并存储 9  
    std::cout << expensiveComputation(3) << std::endl; // 直接返回缓存值 9  
}
```

静态局部变量的应用

3. 状态保持（递归求斐波那契）

```
int fibonacci(int n) {  
    static int memo[100] = {0};  
    if (n <= 1) return n;  
    if (memo[n] != 0) return memo[n];  
    memo[n] = fibonacci(n - 1) + fibonacci(n - 2); // 记忆化搜索  
    return memo[n];  
}
```

表6-1 普通局部变量和静态局部变量的对比

特性	普通局部变量	静态局部变量
作用域	仅在定义它的程序块内可见	仅在定义它的程序块内可见
生命周期	随着程序块执行结束而销毁	贯穿整个程序运行过程
初始化	每次进入程序块都会重新初始化	仅初始化一次，后续调用保持值
默认值	未初始化时值是未定义的	未初始化时值是 0

6.5.2 文件作用域的静态变量

文件作用域的静态变量是定义在函数外部，并使用 `static` 关键字修饰的变量。

```
#include <iostream>

static int fileVar = 10; // 文件作用域的静态变量，只能在当前文件访问

void printVar() {
    std::cout << "fileVar = " << fileVar << std::endl;
}

int main() {
    printVar();
    return 0;
}
```

- 它的 **生命周期** 与普通全局变量相同，**作用域** 限于当前编译单元（即当前 .cpp 文件），而普通全局变量可以被其他文件访问。

```
#include <iostream>

static int fileVar = 10; // 文件作用域的静态变量，只能在当前文件访问

void printVar() {
    std::cout << "fileVar = " << fileVar << std::endl;
}

int main() {
    printVar();
    return 0;
}
```



```
int globalInt;          // 普通全局变量，默认初始化为 0  
static int staticInt; // 静态文件变量，默认初始化为 0
```

```
double globalDouble;    // 默认初始化为 0.0  
static double staticDouble; // 默认初始化为 0.0
```

```
bool globalBool;        // 默认初始化为 false  
static bool staticBool; // 默认初始化为 false
```

```
int* globalPtr;          // 默认初始化为 nullptr  
static int* staticPtr; // 默认初始化为 nullptr
```

- 文件作用域的静态变量和普通全局变量在 默认初始化 上完全相同：

所有具有静态存储期的变量（如全局变量、文件作用域的静态变量、局部静态变量、类的静态成员变量）都遵循相同的 零初始化 规则：

变量类型	默认初始化值
整数类型 (<code>int</code> , <code>long</code> , <code>char</code> , <code>bool</code> 等)	<code>0</code> (<code>bool</code> 是 <code>false</code>)
浮点类型 (<code>float</code> , <code>double</code>)	<code>0.0</code>
指针类型	<code>nullptr</code>
类类型（如果有默认构造函数）	调用默认构造函数初始化
类类型（没有默认构造函数）	编译错误（必须手动初始化）

文件作用域的静态变量 vs. 普通全局变量

特性	普通全局变量	文件作用域的静态变量
作用域	整个程序（所有文件）	仅限当前文件
其他文件是否可访问	✅ 可以使用 <code>extern</code> 访问	❌ 不能被 <code>extern</code> 访问
生命周期	程序运行期间一直存在	程序运行期间一直存在
适用场景	需要跨文件共享的全局数据	仅当前文件使用的全局数据

(1) 使用普通全局变量

file1.cpp

```
#include <iostream>

int globalVar = 100; // 普通全局变量（可在其他文件访问）

void printGlobal() {
    std::cout << "globalVar = " << globalVar << std::endl;
}
```

file2.cpp

```
#include <iostream>

extern int globalVar; // 声明外部变量

int main() {
    std::cout << "Accessing globalVar from file2.cpp: " << globalVar << std::endl;
    return 0;
}
```

(2) 使用文件作用域的静态变量

file1.cpp

```
#include <iostream>

static int staticVar = 200; // 仅限 file1.cpp 可见

void printStatic() {
    std::cout << "staticVar = " << staticVar << std::endl;
}
```

file2.cpp

```
#include <iostream>

extern int staticVar; // ❌ 编译错误, staticVar 不能被 extern 访问

int main() {
    std::cout << "staticVar = " << staticVar << std::endl;
    return 0;
}
```

➡ 编译时会报错:

```
error: 'staticVar' was not declared in this scope
```

文件作用域的静态变量只能在定义它的文件内访问，起到 隐藏全局变量 的作用，不但可以防止名字冲突，还可以保护数据的完整和安全。

file1.cpp

```
#include <iostream>

static int staticVar = 200; // 仅限 file1.cpp 可见

void printStatic() {
    std::cout << "staticVar = " << staticVar << std::endl;
}
```

6.6 函数重载

- 6.6.1 什么是函数重载?
- 6.6.2 什么是参数列表不同?
- 6.6.3 重载解析
- 6.6.4 const对象的 引用或指针对重载的影响

6.6.1 什么是函数重载？

- 函数重载 是指在同一作用域中定义多个函数名相同，但 参数列表不同的函数。这些函数可以具有不同的参数个数或类型，但它们的返回值类型可以相同或不同。

```
int f() { return 0; }           // 无参
int f(int x) { return x; }      // 一个 int 参数
int f(int x, int y) { return x + y; } // 两个 int 参数
double f(double x) { return x * 2; } // 一个 double 参数
```


6.6.1 什么是函数重载？

- 返回类型不是区分重载的依据，因此即使返回类型不同，但参数列表相同也会引起 重定义错误。

错误示例：

```
int f(int x);    // 声明 f 函数
double f(int x); // 错误：重定义，仅返回类型不同
```

6.6.2 什么是参数列表不同？

- 要实现函数重载，必须保证 参数列表不同。参数列表不同的几种情况包括：

- 参数个数不同:

```
void g();           // 无参  
void g(int);        // 一个 int 参数  
void g(int, double); // 一个 int 参数和一个 double 参数
```

- 参数类型不同:

```
void h(int);  
void h(double);  
void h(const char*);
```

- 参数顺序不同:

```
void k(int, double);  
void k(double, int);
```

- 参数为引用或指针的 `const` 修饰符不同:

```
void m(int&);  
void m(const int&);
```

或

```
void n(int*);  
void n(const int*);
```

注意:

- 参数列表不同 不包括 参数的名字。例如:

```
void f(int x);  
void f(int y); // 错误: 重定义, 仅参数名不同
```

- 参数列表不同 不包括 参数是否为 `const` 的区别, 例如:

```
void g(int);  
void g(const int); // 错误: 重定义, 仅 const 修饰不同
```

- 指针类型的 `const` 修饰符 也不影响参数列表，如：

```
void h(int*);
```

```
void h(int* const); // 错误：重定义，仅指针常量化
```

6.6.3 重载解析

当多个同名函数被调用时，编译器会根据 实参 来选择最合适重载函数，这个过程称为 **重载解析**。

```
void f(int) { std::cout << "f(int)\n"; }
void f(double) { std::cout << "f(double)\n"; }
void f(int, double) { std::cout << "f(int, double)\n"; }

int main() {
    f(3);           // 调用 f(int)
    f(3.14);        // 调用 f(double)
    f(3, 3.14);     // 调用 f(int, double)
    f('c');         // 调用 f(int) - char 提升为 int
}
```


重载解析的过程：

1. 确定候选匹配函数集：

- 同名、可见、参数个数匹配、且参数类型可以通过类型转换匹配的函数。

2. 按照参数匹配度选择最佳匹配函数：

- 匹配度优先级：

- 精确匹配：包括无须任何类型转换或仅需平凡转换（如数组到指针、函数到指针、`T` 到 `const T`）。
- 类型提升：例如 `char` 到 `int`、`float` 到 `double`。
- 标准转换：例如 `int` 到 `double`、`int` 到 `unsigned int`、派生类指针到基类指针。
- 自定义类型转换：通过构造函数或类型转换运算符进行转换。

如果存在多个同样匹配度的函数，则匹配失败，编译器会报调用不明确的错误。

6.6.4 const对象的 引用或指针重载的影响

1. const对象的 引用可以绑定到 const 对象、非 const 对象以及字面量：

```
void f(int&);           // 非 const 引用
void f(const int&);     // const 引用

int x = 10;
const int y = 20;
f(x);    // 调用 f(int&)
f(y);    // 调用 f(const int&)
f(30);   // 调用 f(const int&)
```

- 非 const 引用只能绑定到非 const 左值，而 const 引用可以绑定到 任何 类型兼容的值，包括字面量。
- 因此，如果重载了 `int&` 和 `const int&`，字面量或 const 对象会选择 `const int&`。

指向 const对象的指针与 指向非 const对象的指针重载时的区别：

```
void g(int*);           // 指向非const对象的 指针
void g(const int*);     // 指向 const对象 的指针

int x = 10;
const int y = 20;
g(&x);    // 调用 g(int*)
g(&y);    // 调用 g(const int*)
```

- **指向 const 对象的指针** (`const int*`) **可以指向 const 对象或非 const 对象**，但**不能**修改指向对象的值。
- **指向非 const 对象的指针** (`int*`) 只能指向 **非 const 对象**。
- 因此，**指向 const 对象的指针** 在重载时 **适用范围更广**，能够匹配 const 和非 const 对象。

总结

- **函数重载** 通过 **参数列表不同** 来区分同名函数，增强了代码的可读性和灵活性。
- **参数列表不同** 包括：**类型不同、参数顺序不同、const 对象的引用或指针与非 const 对象的引用或指针不同。**
- **返回类型不同不能作为重载的依据**，否则会造成 **重定义错误**。
- **重载解析** 是编译器根据实参来选择最佳匹配函数的过程，包括 **精确匹配、类型提升、标准转换、自定义类型转换** 等。
- **const 对象的引用和指针** 的特殊匹配规则会影响重载解析，包括：
 - `const` 对象的引用可以绑定 **字面量**、`const` 对象、非 `const` 对象。
 - **指向 `const` 对象的指针** 可以指向 `const` 对象或非 `const` 对象。

总结

- 函数重载 通过 参数列表不同 来区分同名函数，增强了代码的可读性和灵活性。
- 参数列表不同 包括：类型不同、参数顺序不同、`const` 对象的引用或指针与非 `const` 对象的引用或指针不同。。
- 返回类型不同不能作为重载的依据，否则会造成 重定义错误。
- ****重载解析**** 是编译器根据实参来选择最佳匹配函数的过程，包括 精确匹配、类型提升、标准转换、自定义类型转换 等。
- `const` 对象的引用和指针 的特殊匹配规则会影响重载解析，包括：
 - `const` 对象的引用可以绑定 字面量、`const` 对象、非 `const` 对象。
 - 指向 `const` 对象的指针 可以指向 `const` 对象或非 `const` 对象。

6.7 内联函数

- 6.7.1. 什么是内联函数?
- 6.7.2 内联函数的工作原理
- 6.7.3 为什么使用内联函数?
- 6.7.4 内联函数的使用场景
- 6.7.5 注意事项与限制
- 6.7.6 内联函数与宏的比较

6.7.1. 什么是内联函数

- C++ 中，函数调用带来一定的性能开销，尤其是在函数体非常短小的情况下，调用所产生的开销可能比函数本身的执行时间还要高。
- 为了降低这种开销，C++ 提供了 **内联函数** (inline function) 的机制，通过将函数的调用在编译时直接替换为函数体代码，从而减少函数调用的额外开销。

6.7.1. 什么是内联函数

- 内联函数 是通过 **inline 关键字** 声明的函数，编译器会在编译时将该函数的调用替换为函数体的代码，从而避免了常规函数调用所产生的开销（例如参数传递、栈帧创建和返回地址保存等）

要定义内联函数，可以在函数声明前使用 `inline` 关键字。比如：

```
inline int add(int a, int b) {  
    return a + b;  
}
```


6.7.2. 内联函数的工作原理

- 当程序编译时，编译器会尝试将所有调用内联函数的地方直接替换为该函数的代码。比如下面的 add 函数调用：

```
int result = add(3, 5);
```

编译器在编译阶段会将这行代码替换为：

```
int result = 3 + 5;
```

这种方式避免了函数调用的开销，直接将计算结果插入调用点。

6.7.3 为什么使用内联函数？

在 C++ 中，函数调用通常需要做一系列的操作，如：

- 参数的传递（可能需要栈操作）
- 栈帧的创建和销毁
- 返回地址的保存和恢复

这些操作对于函数体很短的函数来说，可能是性能的瓶颈，因为这些额外的操作可能比函数体的实际执行时间还要长。

6.7.3 为什么使用内联函数？

内联函数的优势：

- 避免了函数调用的开销，特别是在短小、频繁调用的函数中。
- 减少了函数调用所需的时间和内存开销，能够提高程序的执行效率。

6.7.4 内联函数的使用场景

内联函数适用于那些短小、频繁调用的函数。常见的场景包括：

- 访问器函数（getter/setter）：如获取类成员的函数。
- 简单的数学运算：如加法、乘法等。
- 常量表达式计算。

示例：

```
inline int area(int width, int height) { return width * height; }
```

6.7.5 注意事项与限制

虽然内联函数可以提升性能，但使用时需要注意以下几点：

1. 代码膨胀 (Code Bloat)

如果函数体较大，内联展开会导致代码量显著增加，称为“代码膨胀”。过多的内联可能使程序体积变大，影响指令缓存的命中率，反而降低性能。因此，内联**只适合非常短小的函数**（如一两行的getter/setter或简单运算）。

2. inline **仅是对编译器的建议**

inline 关键字只是向编译器发出“希望内联”的请求，编译器**有权忽略**该请求。对于较复杂的函数（如包含循环、递归或大量语句），编译器通常会拒绝内联，仍按普通函数处理。

3. **递归函数不能内联**

递归函数无法真正内联展开，因为理论上无法预知递归深度。即使标记为 inline，编译器也会忽略，按普通函数调用处理。

4. 类内定义的成员函数默认为内联

在类定义内部直接实现的成员函数（如 class 中的 `int square(int x) { return x*x; }`）会自动被视为内联函数，无需显式写 `inline` 关键字。但编译器是否真的内联，仍遵循第2条规则。

5. 内联函数应定义在头文件中

由于内联函数的展开发生在编译阶段，其完整定义必须在每个调用它的源文件中可见。因此，内联函数通常直接定义在头文件中，而非声明在头文件、定义在源文件。若将内联函数定义在源文件，则其他文件无法看到其定义，无法内联。

6. 调试难度增加

内联展开后，函数调用点被替换为函数体代码，可能使调试器难以单步跟踪函数调用，增加调试难度。开发阶段可暂时去掉 `inline` 以便调试。

6.7.6 内联函数与宏的比较

C++ 中的宏（`#define`）和内联函数在某些方面非常相似，都可以用来避免函数调用的开销。但是，它们有着本质的区别：

- **内联函数**：是由编译器处理的，具有类型检查 and 作用域规则，提供了**更强的安全性**。
- **宏**：是由预处理器在编译之前直接进行替换，**缺乏类型检查**，容易引起意外的错误。

- 宏 (#define) 只是简单的文本替换，没有类型检查，也不会自动处理操作符的优先级，可带来一些潜在的问题。

```
#define SQUARE(x) (x * x) // 宏定义

int main() {
    int a = 5;
    int b = SQUARE(a + 1); // 期望结果是 (5+1) * (5+1) = 36
    std::cout << "b = " << b << std::endl; // 实际输出是 11
    return 0;
}
```

```
#define SQUARE(x) (x * x) // 宏定义

int main() {
    int a = 5;
    int b = SQUARE(a + 1); // 期望结果是 (5+1) * (5+1) = 36
    std::cout << "b = " << b << std::endl; // 实际输出是 11
    return 0;
}
```

解释：

SQUARE(a + 1) 在预处理阶段会被替换成 $(a + 1 * a + 1)$ ，而不是 $((a + 1) * (a + 1))$ 。

由于运算符 $*$ 的优先级高于 $+$ ，最终展开后变成 $a + (1 * a) + 1$ ，即 $5 + 5 + 1 = 11$ ，计算结果完全错误！

如果改用内联函数，则没有任何问题：

```
#include <iostream>

inline int square(int x) { return x * x; }

int main() {
    int a = 5;
    int b = square(a + 1); // 期望结果 36，实际也是 36
    std::cout << "b = " << b << std::endl;
    return 0;
}
```

如果改用内联函数，则没有任何问题：

```
#include <iostream>

inline int square(int x) { return x * x; }

int main() {
    int a = 5;
    int b = square(a + 1); // 期望结果 36，实际也是 36
    std::cout << "b = " << b << std::endl;
    return 0;
}
```

解释：

- `square(a + 1)` 不会进行简单的文本替换，而是正常传递参数 `a+1`，然后计算 `(a + 1) * (a + 1)`，得到正确结果 `36`。
- 这里 `square(a + 1)` 仍然可能被编译器展开为 `((5 + 1) * (5 + 1))`，但不会出现运算优先级错误，因为内联函数遵循 C++ 语言的类型检查和运算规则。

优先考虑 **内联函数**

- 内联函数比宏更加安全，因为它能确保类型正确，而宏没有类型检查，可能会导致意外的结果。因此，在现代 C++ 编程中，应优先考虑 **内联函数**，避免使用宏定义表示函数。

6.7.7 总结

- **内联函数** 是编译时将函数调用替换为函数体的代码，从而减少函数调用的开销。
- 使用 inline 关键字可以提示编译器进行内联，但编译器最终会决定是否进行内联。
- 内联函数适用于短小、频繁调用的函数，但过度使用可能导致 **代码膨胀**，影响性能。
- **内联函数** 和 **宏** 有相似的作用，但内联函数更加安全，具有类型检查功能。

6.8 constexpr

constexpr 是 C++11 引入的关键字，用于声明**常量表达式**——即在编译阶段就能计算出结果的表达式。它可以修饰变量和函数，使计算在编译期完成，从而提升程序性能。

- constexpr **变量**：必须在编译期确定值，且值不可修改。
- constexpr **函数**：若传入的参数是常量表达式，则在编译期计算；否则在运行期计算（仍可作为普通函数使用）。

6.8 constexpr 关键字

- 6.8.1 constexpr 变量
- 6.8.2 constexpr 函数
- 6.8.3 扩展：C++20 constexpr 和 constexpr

6.8.1 constexpr 变量

- constexpr 变量的值必须在编译时确定，且初始化后不可改变。

```
constexpr int x = 10;           // ✓ 编译期常量  
constexpr int y = x + 20;       // ✓ 编译期计算
```

- 初始化表达式必须为常量表达式，否则编译错误：

```
int a = 10;  
constexpr int z = a;           // ✗ a 不是常量表达式
```

const 与 constexpr 变量的对比

- const 变量不一定在编译期确定，其初始化表达式可以是运行期变量；而 constexpr 变量强制编译期确定。

```
const int c1 = 10;           // 编译期常量
const int c2 = rand();       // 运行期常量，值在运行时确定

constexpr int ce1 = 10;      // ✅ 必须编译期确定
// constexpr int ce2 = rand(); // ❌ rand() 不是常量表达式
```

const 与 constexpr 变量的对比

- 只有当一个 const 变量的初始化值是**编译期常量**时，它本身才能被视为编译期常量，并可用于初始化 constexpr 变量。

```
const int a = 10;  
constexpr int d = a;
```

// a 是编译期常量

//  正确: a 是编译期常量

```
int x = 20;  
const int y = x;  
// constexpr int z = y;
```

// y 在运行期才能确定 (因为 x 不是常量)

//  错误: y 不是编译期常量

`const` vs. `constexpr` 变量

关键字	值是否必须在编译期确定	示例
<code>constexpr</code>	必须 是编译期常量	<code>constexpr int a = 10;</code>
<code>const</code>	可能 是编译期或运行期常量	<code>const int b = someFunction();</code>

6.8.2 constexpr 函数

- constexpr 函数可以同时用于编译期和运行期计算。若传入的实参均为**常量表达式**，则在编译期求值；否则在运行期执行普通函数调用。

```
constexpr int square(int x) {  
    return x * x;  
}  
  
int main() {  
    constexpr int a = square(5);           // ✓ 编译期计算  
    int n = 5;  
    int b = square(n);                     // ✓ 运行期计算  
}
```


函数特点

- **双重身份**：既是编译期函数，也是运行期函数。
- **参数决定行为**：是否在编译期执行取决于实参是否为常量表达式。

```
constexpr int add(int a, int b) {  
    return a + b;  
}
```

```
constexpr int sum = add(3, 4);           // 编译期计算  
int x = 3, y = 4;  
int runtime_sum = add(x, y);             // 运行期计算
```

限制 (C++14 放宽后)

- **C++11**: 函数体只能包含一条 `return` 语句。
- **C++14 起**: 可包含局部变量、循环、分支等, 但仍不能出现:
 - 动态内存分配 (`new/delete`)
 - 异常处理 (`try/catch/throw`)
 - 调用非 `constexpr` 函数
 - 虚函数调用
 - `goto` 语句

C++14 及以后示例：编译期计算阶乘

```
constexpr int factorial(int n) {  
    int result = 1;  
    for (int i = 2; i <= n; ++i)  
        result *= i;  
    return result;  
}
```

```
constexpr int fac5 = factorial(5);    // 120, 编译期计算
```

典型应用

- **常量计算**: 数学公式、查找表生成
- **数组大小**: `int arr[factorial(4)];`
- **模板参数**: 作为非类型模板参数
- **编译期字符串处理**: 结合 `std::string_view` 实现编译期解析

6.8.3 C++20 扩展: `constexpr` 与 `constexpr`

C++20 进一步细化了编译期计算的能力，引入了两个新关键字。

constexpr : 强制编译期函数

constexpr 修饰的函数**只能在编译期执行**，若在运行期调用则编译错误。它比 constexpr 更严格，适合那些必须编译期完成的逻辑。

```
constexpr int cube(int x) {  
    return x * x * x;  
}
```

```
constexpr int a = cube(3);
```

//  编译期计算

```
int b = 3;
```

```
// int c = cube(b);
```

//  错误: b 不是常量表达式

constinit : 保证编译期初始化

- constinit 用于修饰**静态存储期**的变量（全局变量、静态局部变量），强制其在编译期初始化。初始化后变量可以修改（不同于 const）。

```
constinit int global = 100;           // ✓ 编译期初始化
constinit static int local = 42;     // ✓ 静态局部变量也可用

int main() {
    global = 200;                     // ✓ 运行时可修改
    // constinit int x;              // ✗ 不能用于普通局部变量
}
```

constinit : 保证编译期初始化

- 若初始化式不是常量表达式，则编译错误：

```
int square(int x) { return x * x; }  
int x = 5;  
constinit int y = square(5);    // ✅ square(5) 是常量表达式  
// constinit int z = square(x); // ❌ x 不是常量表达式
```


6.8.4 constexpr、constexpr、constexpr 对比

关键字	适用对象	执行时机	是否可变	主要用途
constexpr	变量/函数	编译期或运行期	变量只读	通用的编译期计算
constexpr	函数	仅编译期	函数只读	强制编译期执行
constexpr	静态/全局变量	编译期初始化	可变	确保初始化顺序，避免动态初始化开销

- `constexpr` 用于修饰**函数**，强制该函数只能在编译期求值，不能在运行时调用。
- `constexpr` 用于修饰**变量**（必须是具有静态或线程存储期的变量，如全局变量、静态局部变量等），它确保变量在编译期初始化，但变量本身**不一定是常量**——初始化后，可以在运行时修改它（除非同时加了`const`）。
- `constexpr` 既可以修饰变量也可以修饰函数。修饰变量时，该变量必须是编译期常量，且值不可修改。
- `constexpr` 变量可以是全局变量，也可以是静态局部变量。

// constexpr 全局变量：必须在编译期初始化，且不可修改

```
constexpr int globalConstexpr = 100;
```

```
void example() {
```

// constexpr 静态局部变量：仅在第一次进入函数时编译期初始化，之后值保持不变

```
static constexpr int staticLocalConstexpr = 42;
```

// constinit 静态局部变量：必须在编译期初始化，但可以在运行时修改

```
static constinit int staticLocalConstinit = 100;
```

```
staticLocalConstinit++; // ✅ 运行时修改
```

```
}
```

// constexpr 函数：强制编译期执行

```
constexpr int square(int x) {
```

```
    return x * x;
```

```
}
```

```
int main() {
```

```
    constexpr int a = square(5); // ✅ 编译期计算
```

```
    // int b = square(some_runtime_var); // ❌ 错误
```

```
}
```

选择建议：

- 需要编译期确定的数值、数组大小、模板参数 → `constexpr` 变量
- 编写可同时用于编译期和运行期的通用函数 → `constexpr` 函数
- 必须编译期执行（如模板元编程） → `constexpr` 函数
- 静态/全局变量需编译期初始化且后续可修改 → `constexpr` 变量

6.9 递归函数：调用自身的函数

6.9.1 递归和递归函数

6.9.2 阶乘的递归函数

6.9.3 递归函数的基本结构

6.9.4 斐波那契数列

6.9.5 最大公约数

6.9.6 汉诺塔问题

6.9.7 快速排序算法

6.9.8 迷宫问题

6.9.9 小结

6.9.1 递归和递归函数

- 递归是一个任务分解的解决问题的方法，一个大的问题如果能够分解成和它类似的子问题，且子问题的解决方法和大问题是一样的，只不过问题的规模有所区别而已。这种情况下就可以采用递归的方法来解决这个问题。
- 递归函数是指在函数内部调用自身的函数。例如，计算 **n 的阶乘** 就是一个典型的递归问题。

$$n! = n * (n-1)!$$

6.9.2 阶乘的递归函数

- 大问题（ n 的阶乘）被分解为更小的子问题（ $(n-1)$ 的阶乘）。

阶乘 是指从 1 乘到 n 的积，记作 $n!$ ，且满足以下关系：

- $n! = n \times (n - 1)!$
- 当 $n = 1$ 时， $1! = 1$ （这称为基情形，即递归终止的条件）


```
#include <iostream>
int fact(int n) {
    if (n == 1) // 基情形: 如果 n 等于 1, 就直接返回 1
        return 1;
    return n * fact(n - 1); // 递归调用: n * (n-1)!
}
int main() {
    std::cout << fact(4) << '\n'; // 输出: 24
}
```

```
int main() {  
    std::cout << fact(4) << '\n';    //输出:  24  
}
```

递归是一个嵌套的过程，如 `fact(4)` 的递归计算过程如下：

```
fact(4)  
4 * fact(3)  
4 * (3 * fact(2))  
4 * (3 * (2 * fact(1)))  
4 * (3 * (2 * 1 ))  
4 * (3 * 2)  
4 * 6  
24
```

6.9.3 递归函数的基本结构

递归函数通常包含两部分：

1. **基情形 (Base Case)**：递归终止条件，避免无限递归。例如，在阶乘函数中，当 $n == 1$ 时直接返回 1。
2. **递归情形 (Recursive Case)**：将问题分解为更小的子问题，并调用自身进行求解。例如， $n * \text{fact}(n - 1)$ 。

6.9.3 递归函数的基本结构

编写递归函数时，一定要确保：

- 基情形明确且可以终止递归。
- 每次递归调用都将问题规模缩小，最终能到达基情形。

6.9.4 斐波那契数列

$f(0) = 1, f(1) = 1, \text{当 } n \geq 2 \text{ 时}, f(n) = f(n - 1) + f(n - 2)$

```
#include <iostream>
int fib(int n) {
    if (n <= 2)                //基情形
        return 1;
    else                        //递归情形
        return fib(n - 1) + fib(n - 2);
}
int main() {
    for (int i{1};i!=8;i++)
        std::cout << fib(i) << '\t';
}
```

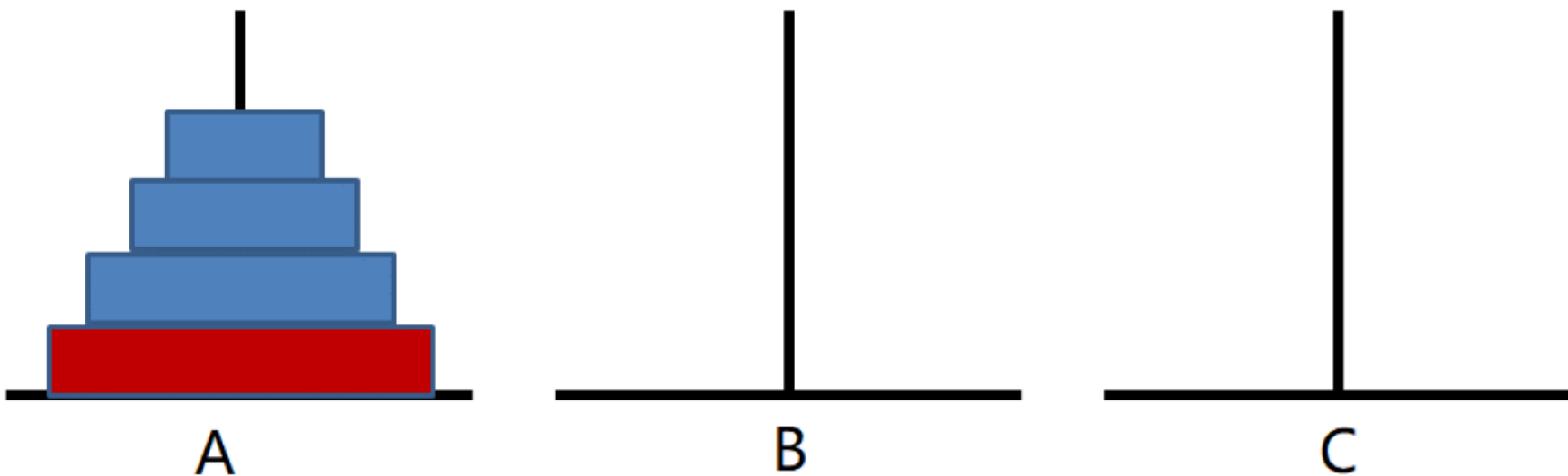
• 6.9.5 最大公约数

$$gcd(m, n) = \begin{cases} m & n = 0 \\ gcd(n, m \% n) & n \neq 0 \end{cases}$$

```
#include <iostream>
int gcd(int m, int n) {
    if (n == 0)
        return m;
    else
        return gcd(n, m%n);
}
int main() {
    std::cout << gcd(72,27) << '\t' << gcd(24, 36) << '\t';
}
```

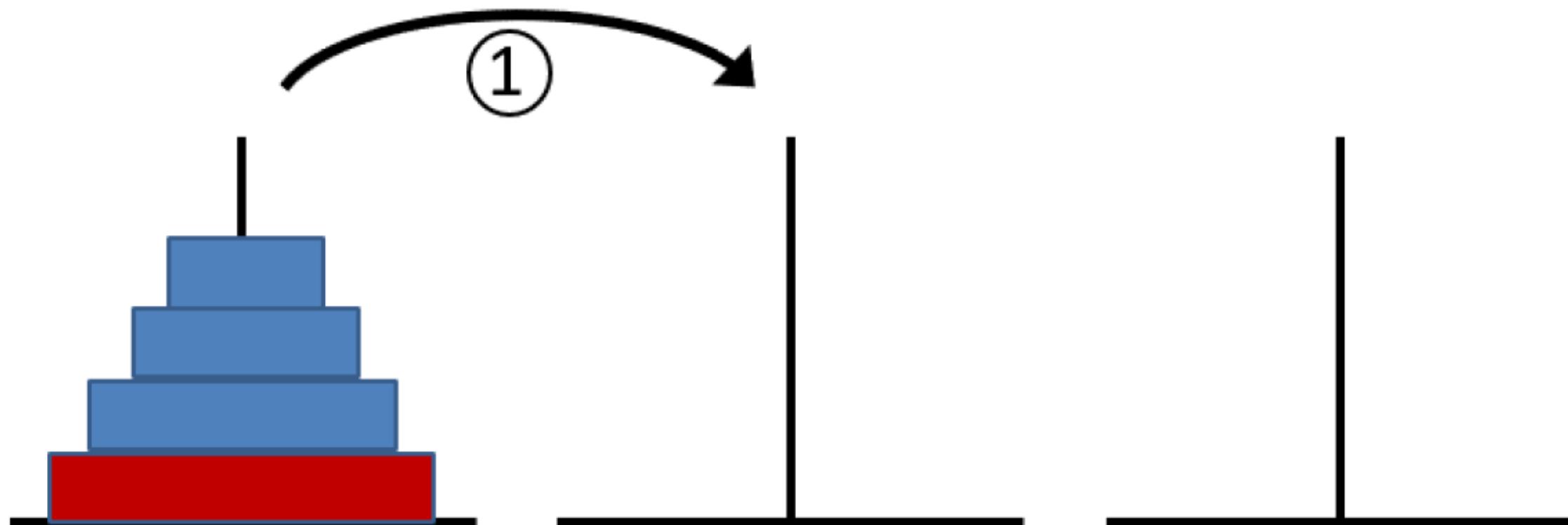
6.9.6 汉诺塔问题

- 有三根柱子A、B、C，其中A柱子上有N个 ($N > 1$) 盘子，盘的尺寸从下到上依次变小。现在要求将盘子全部移到C塔，每次只能移动一个盘子，且小盘必须在大盘之上，当然，盘子只能放在在3个柱子之一上。

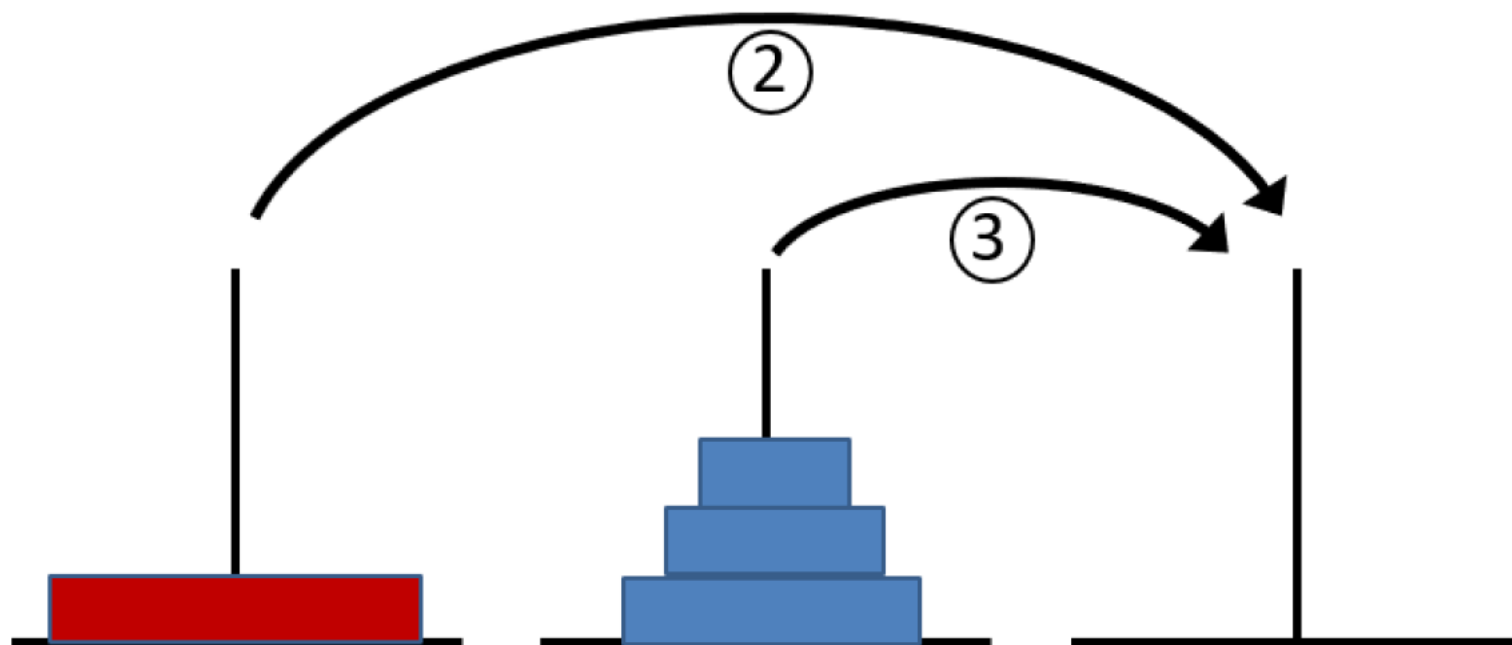


如果采用“分而治之”的解决问题方法，可以将“N个盘子的移动（从A柱借助于B柱移到C柱）”分解为如下3个子问题（如图6-2所示）。

- （上面的）N-1个盘子的移动(从A柱借助于C柱移到B柱)
- 最大盘子的移动: 直接从A柱移到C柱
- N-1个盘子的移动(从B柱借助于A柱移到C柱)



N-1个盘子的移动(从B柱借助于A柱移到C柱)

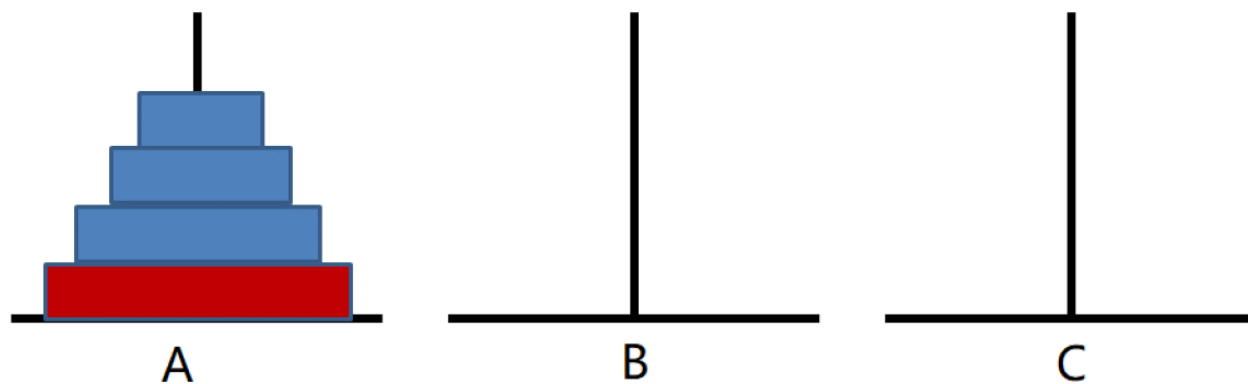


// 一个盘子：直接移动

```
void moveDisk(int i, const char x, const char y) {  
    std::cout << "moving disk" << i << " from " << x << " to " << y << '\n';  
}
```

// 参数：盘数，起始柱，中转柱，目标柱

```
void move(int n, const char a, const char b, const char c) {  
    if (n < 1) return;  
    move(n - 1, a, c, b); // n - 1个盘子从A柱借助于C柱移到B柱  
    moveDisk(n, a, c);  
    move(n - 1, b, a, c); // n - 1个盘子从B柱借助于A柱移到C柱  
}  
  
int main() {  
    move(3, 'A', 'B', 'C');  
}
```



执行程序的输出为：

```
moving disk 1  from A to C
moving disk 2  from A to B
moving disk 1  from C to B
moving disk 3  from A to C
moving disk 1  from B to A
moving disk 2  from B to C
moving disk 1  from A to C
```

6.9.7 快速排序算法

34, 2, 89, 47, 29, 13



“一次划分”

2, 29, 13, [34], 89, 47

// qsort是对[start, end]区间的元素进行快速排序过程

```
void qsort(T arr[], const int start, const int end) {
```

```
    if (start >= end) return;
```

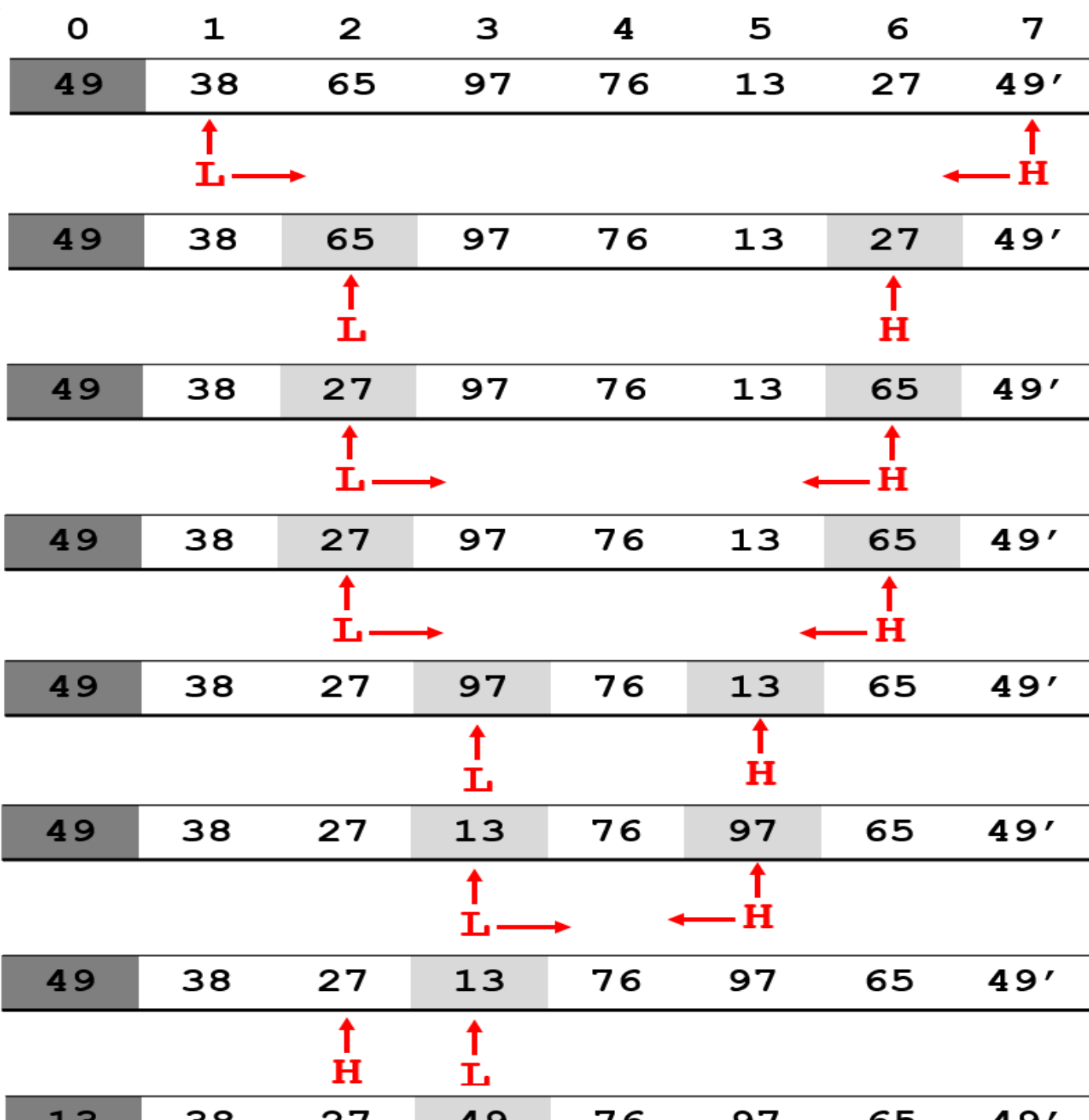
```
    // partition将[start, end]之间的序列一次划分为2部分, 返回pivot是基准的位置
```

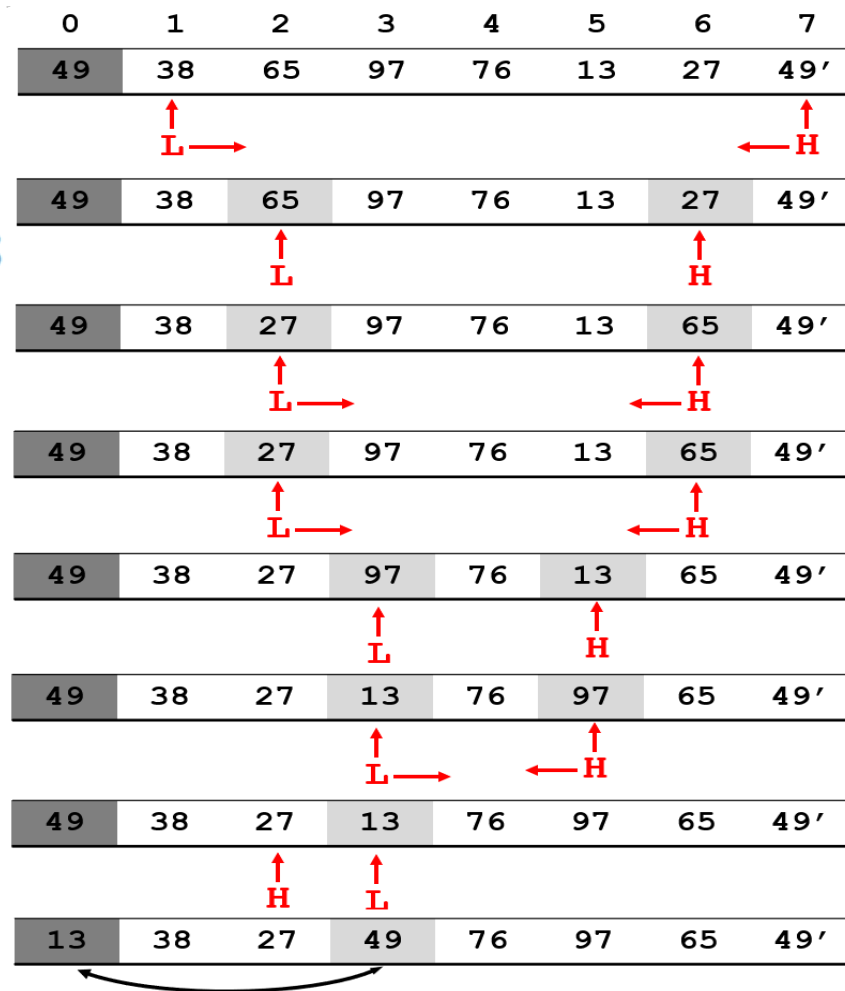
```
    auto pivot = partition(arr, start, end); //先对原序列一次划分
```

```
    qsort(arr, start, pivot - 1); //对[start, pivot - 1]之间的序列调用qsort快速排序过程
```

```
    qsort(arr, pivot + 1, end); //对[pivot + 1, end]之间的序列调用qsort快速排序过程,
```

```
}
```



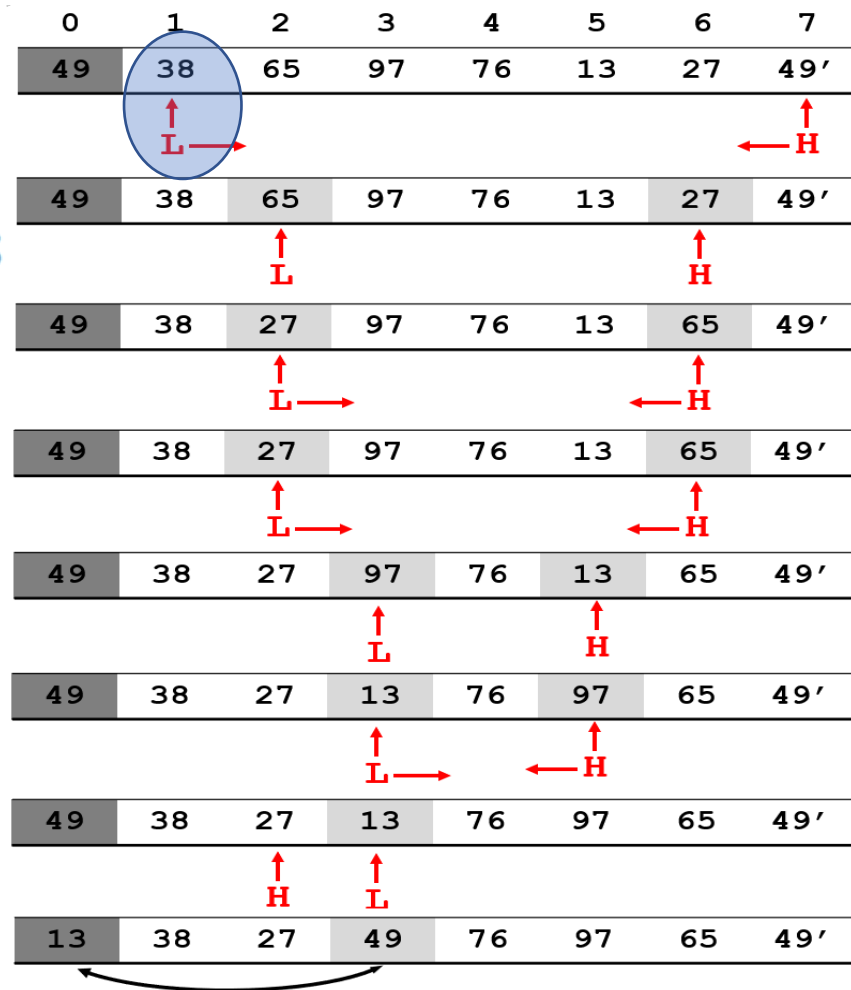



```

int partition(T arr[], const int start, const int end) {
    auto pivotvalue{ arr[start] };
    auto L = start + 1, R = end;

    auto done{ false };
    while (!done) {
        while (L <= R and arr[L] <= pivotvalue) L = L + 1;
        while (arr[R] >= pivotvalue and R >= L) R = R - 1;
        if (R < L) done = true;
        else { //交换L,R的值
            auto temp{ arr[L] };
            arr[L] = arr[R];
            arr[R] = temp;
        }
    }
    //交换R和start的值, 将基准元素移到基准位置R
    auto temp{ arr[R] };
    arr[R] = arr[start];
    arr[start] = temp;
    return R;
}

```

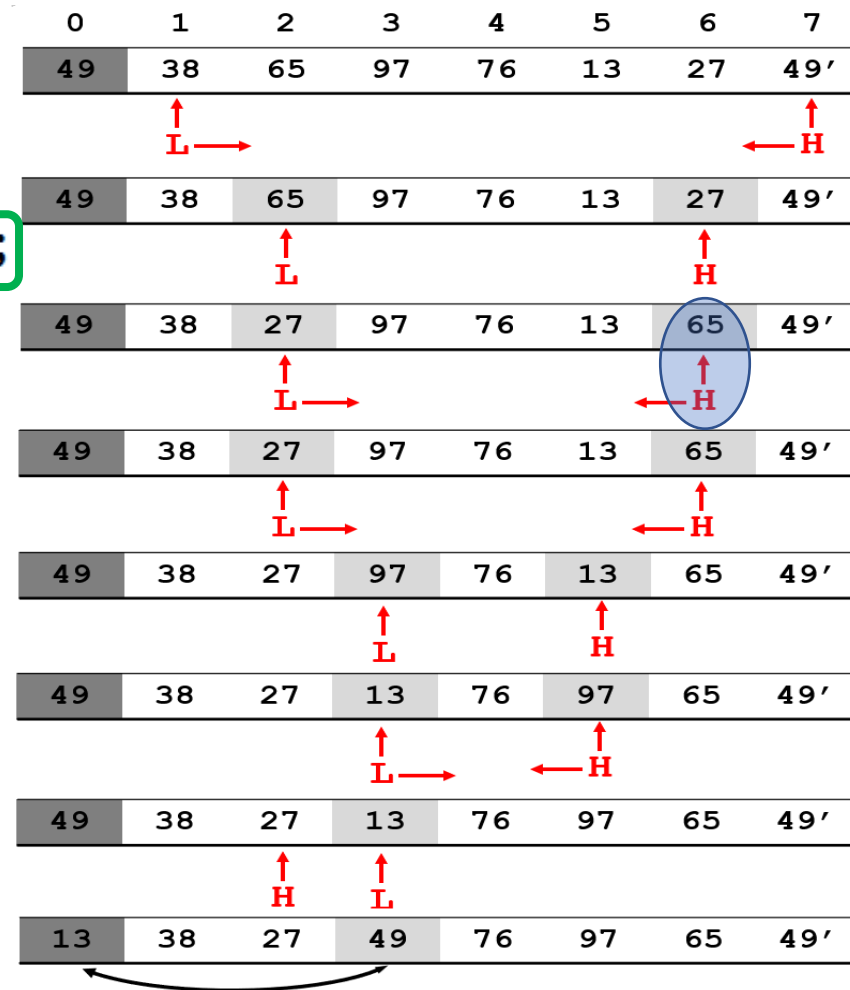



```

int partition(T arr[], const int start, const int end) {
    auto pivotvalue{ arr[start] };
    auto L = start + 1, R = end;

    auto done{ false };
    while (!done) {
        while (L <= R and arr[L]<=pivotvalue) L = L + 1;
        while (arr[R] >= pivotvalue and R >= L) R = R - 1;
        if (R < L) done = true;
        else { //交换L,R的值
            auto temp{ arr[L] };
            arr[L] = arr[R];
            arr[R] = temp;
        }
    }
    //交换R和start的值, 将基准元素移到基准位置R
    auto temp{ arr[R] };
    arr[R] = arr[start];
    arr[start] = temp;
    return R;
}

```

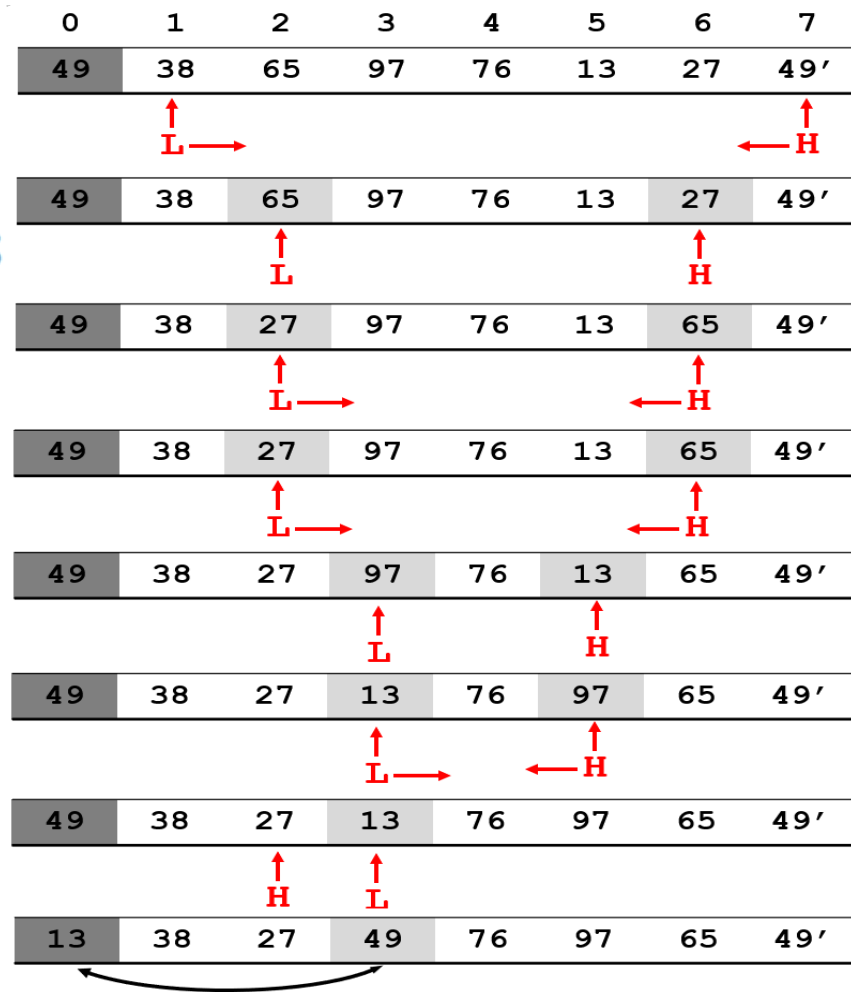


```

int partition(T arr[], const int start, const int end) {
    auto pivotvalue{ arr[start] };
    auto L = start + 1, R = end;

    auto done{ false };
    while (!done) {
        while (L <= R and arr[L]<=pivotvalue) L = L + 1;
        while (arr[R] >= pivotvalue and R >= L) R = R - 1;
        if (R < L) done = true;
        else { //交换L,R的值
            auto temp{ arr[L] };
            arr[L] = arr[R];
            arr[R] = temp;
        }
    }
    //交换R和start的值, 将基准元素移到基准位置R
    auto temp{ arr[R] };
    arr[R] = arr[start];
    arr[start] = temp;
    return R;
}

```

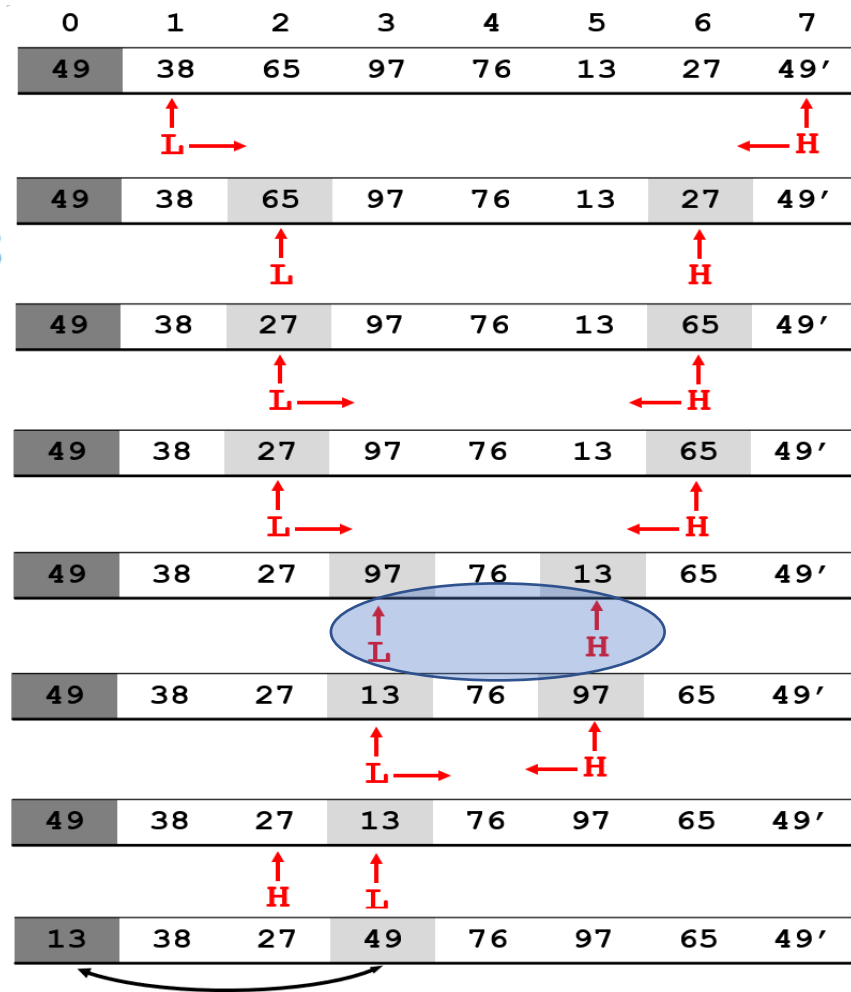


```

int partition(T arr[], const int start, const int end) {
    auto pivotvalue{ arr[start] };
    auto L = start + 1, R = end;

    auto done{ false };
    while (!done) {
        while (L <= R and arr[L]<=pivotvalue) L = L + 1;
        while (arr[R] >= pivotvalue and R >= L) R = R - 1;
        if (R < L) done = true;
        else { //交换L,R的值
            auto temp{ arr[L] };
            arr[L] = arr[R];
            arr[R] = temp;
        }
    }
    //交换R和start的值, 将基准元素移到基准位置R
    auto temp{ arr[R] };
    arr[R] = arr[start];
    arr[start] = temp;
    return R;
}

```



```

int partition(T arr[], const int start, const int end) {
    auto pivotvalue{ arr[start] };
    auto L = start + 1, R = end;

    auto done{ false };
    while (!done) {
        while (L <= R and arr[L]<=pivotvalue) L = L + 1;
        while (arr[R] >= pivotvalue and R >= L) R = R - 1;
        if (R < L) done = true;
        else { //交换L,R的值
            auto temp{ arr[L] };
            arr[L] = arr[R];
            arr[R] = temp;
        }
    }
}

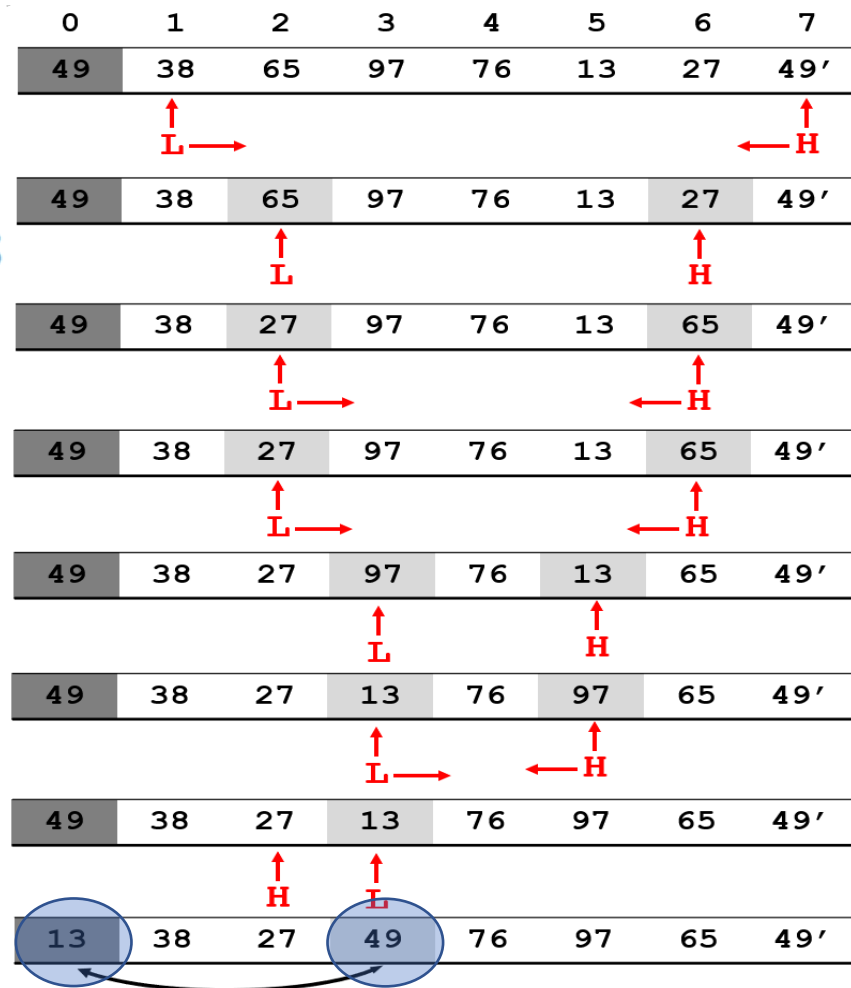
```

//交换R和start的值, 将基准元素移到基准位置R

```

auto temp{ arr[R] };
arr[R] = arr[start];
arr[start] = temp;
return R;

```

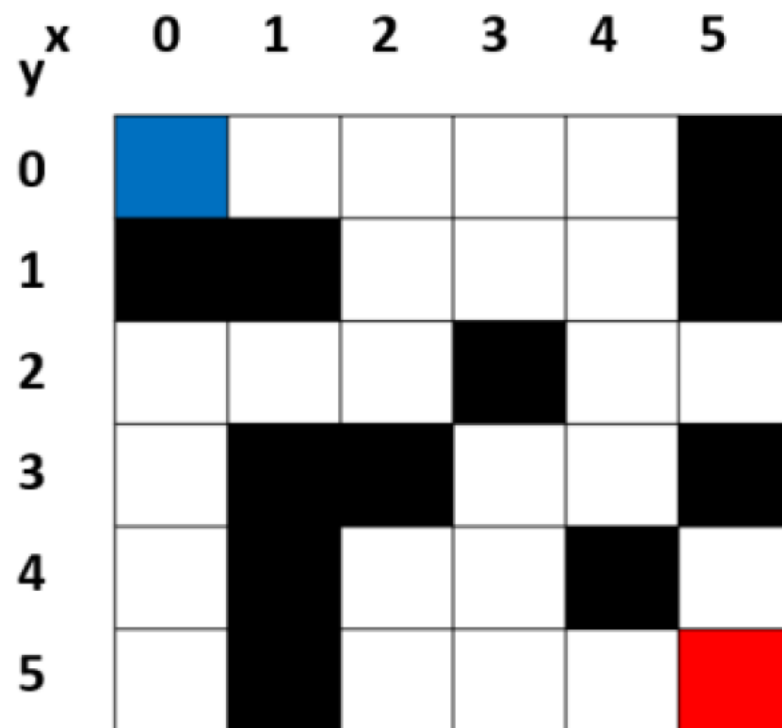


```
void quickSort(T arr[], const int n) { //对n个元素的数组arr的快速排序  
    qsort(arr, 0, n - 1); //调用对一个区间快速排序过程qsort  
}
```

```
#include <iostream>  
int main() {  
    int a[] { 49, 38, 27, 97, 76, 13, 27, 49 };  
    quickSort(a, 8);  
    for (auto i { 0 }; i != 8; i++)  
        std::cout << a[i] << '\t';  
    std::cout << '\n';  
}
```

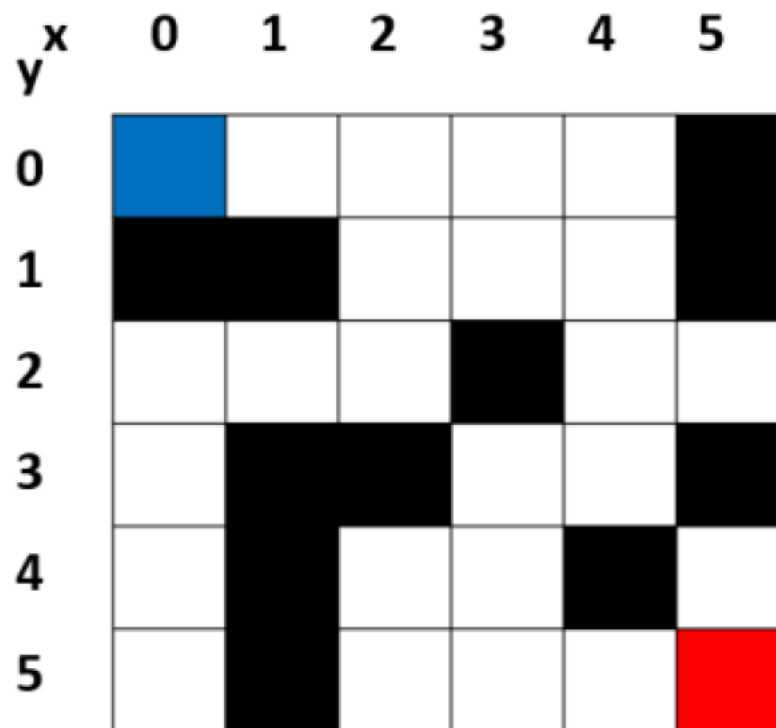
6.9.8 实战：迷宫问题

- 给定一个迷宫，指明起点和终点，找出从起点出发到终点的有效可行路径，就是迷宫问题（maze problem）。可行路径是指不经过障碍物、不重复经过同一位置的路径。如图6是一个迷宫。



- 迷宫可以用二维数组来表示。0表示通路，1表示障碍，2表示终点。

```
maze = [ [0, 0, 0, 0, 0, 1],  
          [1, 1, 0, 0, 0, 1],  
          [0, 0, 0, 1, 0, 0],  
          [0, 1, 1, 0, 0, 1],  
          [0, 1, 0, 0, 1, 0],  
          [0, 1, 0, 0, 0, 2]]
```



- 迷宫求解问题可以描述为一个递归过程：对于一个当前位置，判断该位置是否为终点（值为2）、墙（值为1）、已经走过（值为3）。如果不是上述情况，说明该位置可通但未走过（值为0），可从该位置走向其4邻（上、下、左、右4个位置），对新位置重复这个过程。
- 为了避免死循环，需要将走过的位置标记为3。

其伪代码算法如下：

```
solveMaze(x, y)
    //基情形
    if(该位置(x,y)是终点 (2) ) 返回 true;
    else if (该位置(x,y)是墙 (1)、已经走过 (3))
        返回 false。
    //递归探索：尝试向上、向下、向左、向右移动：
    if (solveMaze(x-1, y) )    返回 true;
    if (solveMaze(x+1, y) )    返回 true;
    if (solveMaze(x, y-1) )    返回 true;
    if (solveMaze(x, y+1) )    返回 true;
    返回 false
}
```

```
bool go_maze(int maze[][6], const int x, const int y, const int n = 6) {  
    //该位置(x,y)是否终点(2)、墙(1)、已经走过(3)  
    if (maze[x][y] == 2) {  
        std::cout << "到达终点: " << x << ", " << y << '\n';  
        return true;  
    }  
    else if (maze[x][y] == 1 or maze[x][y] >= 3) //墙或已经走  
        return false;  
  
    //从该位置向4邻探索  
    std::cout << "访问: " << x << ", " << y << '\n';  
    maze[x][y] = 3;           // 标记该位置已经访问过  
    // 向4邻探索  
    if ((x < n - 1 and go_maze(maze, x + 1, y))  
        or (y > 0 and go_maze(maze, x, y - 1))  
        or (x > 0 and go_maze(maze, x - 1, y))  
        or (y < n - 1 and go_maze(maze, x, y + 1)))  
        return true;  
    maze[x][y] = 4;           //此位置不通  
    return false;  
}
```

```
void print(int maze[][6], const int n = 6) {  
    for (auto i = 0; i < n; i++) {  
        for (auto j = 0; j < n; j++) {  
            std::cout << maze[i][j] << ' ';  
        }  
        std::cout << "\n";  
    }  
}
```

在主函数里调用上述走迷宫函数。

```
int main() {  
    int maze[][6] = {  
        {0, 0, 0, 0, 0, 1},  
        {1, 1, 0, 0, 0, 1},  
        {0, 0, 0, 1, 0, 0 },  
        {0, 1, 1, 0, 0, 1},  
        {0, 1, 0, 0, 1, 0 },  
        {0, 1, 0, 0, 0, 2} };  
    go_maze(maze, 0, 0);  
    print(maze);  
}
```

访问: 0,0
访问: 0,1
访问: 0,2
访问: 1,2
访问: 2,2
访问: 2,1
访问: 2,0
访问: 3,0
访问: 4,0
访问: 5,0
从5,0回退
从4,0回退
从3,0回退
从2,0回退
从2,1回退
从2,2回退
访问: 1,3

访问: 0,3
访问: 0,4
访问: 1,4
访问: 2,4
访问: 3,4
访问: 3,3
访问: 4,3
访问: 5,3
访问: 5,2
访问: 4,2
从4,2回退
从5,2回退
访问: 5,4
到达终点: 5,5
3 3 3 3 3 1
1 1 3 3 3 1
3 3 3 1 3 0
3 1 1 3 3 1
3 1 3 3 1 0
3 1 3 3 3 2

```
[ [0, 0, 0, 0, 0, 1],  
  [1, 1, 0, 0, 0, 1],  
  [0, 0, 0, 1, 0, 0],  
  [0, 1, 1, 0, 0, 1],  
  [0, 1, 0, 0, 1, 0],  
  [0, 1, 0, 0, 0, 2]]
```

6.9.9 小结

- **递归函数** 是调用自身的函数，通过任务分解来解决问题。
- 每个递归函数都包含两部分：
 - **基情形 (Base Case)**：用于终止递归，防止无限调用。
 - **递归情形 (Recursive Case)**：将问题规模缩小，逐步向基情形靠拢。
- 递归过程体现了 **嵌套调用** 的特点，每次调用都会保存当前状态并等待下一层返回结果。

- 使用递归时要注意：
 - 基情形必须明确，否则会导致 **无限递归** 和 **栈溢出**。
 - 问题规模应逐步缩小，以确保最终到达基情形。
 - **递归嵌套过深可能导致程序堆栈溢出**，应根据问题规模和系统栈空间大小合理设计递归深度。
 - 对于重复计算较多的递归（如斐波那契数列），应考虑 **记忆化** 或 **动态规划** 优化。

6.10 函数指针

- C++ 中，**函数也是一种数据**，它在内存中占据一块固定的区域。因此，我们可以通过 **指针** 来访问和调用函数。这种**指向函数的指针**称为 **函数指针**。
- 函数指针可以用于 **回调函数**、**函数数组**、**动态调用不同的函数** 等场景，使程序更加灵活。

6.10 函数指针

- 6.10.1 计算器函数calculate
- 6.10.2 函数指针的定义和初始化
- 6.10.3 通过函数指针调用函数
- 6.10.4 函数指针作为函数参数
- 6.10.5 函数指针数组

6.10.1 从计算器例子看函数指针的需求

假设已有加法和减法函数：

```
int add(int a, int b) { return a + b; }  
int subtract(int a, int b) { return a - b; }
```

若编写一个通用计算函数，传统方式需要根据运算符判断调用哪个函数：

```
void calculate(char op, int x, int y) {  
    if (op == '+') std::cout << add(x, y);  
    else if (op == '-') std::cout << subtract(x, y);  
    else std::cout << "无效运算符";  
}
```

这种方法每增加一种运算就要修改 calculate , 不利于维护。

```
void calculate(char op, int x, int y) {  
    if (op == '+') std::cout << add(x, y);  
    else if (op == '-') std::cout << subtract(x, y);  
    else std::cout << "无效运算符";  
}
```

使用函数指针可以解耦运算与选择逻辑：

```
void calculate(int (*op)(int, int), int x, int y) {  
    std::cout << op(x, y);  
}  
  
int main() {  
    calculate(add, 5, 3);           // 输出8  
    calculate(subtract, 5, 3);      // 输出2  
}
```

这里 **int (*op)(int, int)** 是一个函数指针，它可以指向任何"接受两个 int 并返回 int "的函数。

6.10.2 函数指针的定义和初始化

函数指针的定义方式如下：

```
返回值类型 (*指针名)(参数类型列表);
```

例如：

```
int (*funcPtr)(int, int);
```

- 表示 funcPtr 是一个**指向返回值类型为 int**，并且有两个 int 类型参数的**函数指针**。

函数指针的初始化

- 函数指针的初始化方式是**将函数名赋值给它**，例如：

```
funcPtr = add;
```

注意：函数名本身就是指向该函数的指针，不需要 & 取地址符号，因此 `funcPtr = add;` 是正确的写法。

6.10.3 通过函数指针调用函数

函数指针调用函数的方式有两种：

```
int result = funcPtr(5, 3);           // 直接调用（推荐）  
int result = (*funcPtr)(5, 3);        // 显式解引用后调用
```

两者等价，因为函数指针会被自动解引用。

函数指针调用函数的方式有两种：

```
int result = funcPtr(5, 3);           // 直接调用（推荐）  
int result = (*funcPtr)(5, 3);        // 显式解引用后调用
```

```
#include <iostream>  
  
int multiply(int a, int b) { return a * b; }  
  
int main() {  
    int (*op)(int, int) = multiply;  
    std::cout << op(4, 5) << '\n';    // 20  
    std::cout << (*op)(4, 5) << '\n'; // 20  
}
```


6.10.4 函数指针作为函数参数

- 函数指针可以作为函数的参数传递，使函数能够调用不同的运算逻辑。

请注意，`calculate` 函数的第一个形参是函数指针：

```
void calculate(int (*op)(int, int), int x, int y)
```

传递不同的函数指针给它，如 `calculate(add, 5, 3)` 和 `calculate(subtract, 5, 3)`，从而实现不同的运算。

6.10.5 函数指针数组

函数指针数组可以将多个函数组织起来，通过索引动态选择调用。
例如，实现一个简易计算器：

```
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }

int main() {
    // 定义函数指针数组
    int (*operations[])(int, int) = { add, subtract, multiply };
```

```
int (*operations[])(int, int) = { add, subtract, multiply };
char ops[] = { '+', '-', '*' };

int x, y;
char op;
std::cout << "请输入计算表达式（格式：a + b）：";
std::cin >> x >> op >> y;

// 查找操作符对应的索引
int index = -1;
for (int i = 0; i < 3; ++i) {
    if (ops[i] == op) {
        index = i;
        break;
    }
}
```

6.11 Lambda 表达式

Lambda 表达式是 C++11 引入的一种匿名函数形式，它允许我们在需要函数的地方快速定义一段简短的、一次性的函数逻辑。

Lambda 特别适用于回调函数、STL 算法中的自定义操作以及事件驱动的场景。

6.11 Lambda 表达式

- 6.11.1 为什么需要 Lambda 表达式?
- 6.11.2 Lambda 表达式的基本形式和语法
- 6.11.3 Lambda 表达式的赋值
- 6.11.4 Lambda 的参数和返回值
- 6.11.4 捕获外部变量
- 6.11.5 可变 Lambda (mutable)
- 6.11.6 初始化捕获 (init capture)
- 6.11.7 Lambda 表达式的应用场景

6.11.1 为什么需要 Lambda 表达式?

- 自定义排序规则

```
// 自定义比较函数
bool compare(int a, int b) {
    return a > b; // 降序排序
}

int main() {
    std::vector<int> numbers{ 3, 1, 4, 1, 5, 9, 2, 6 };

    // 使用 compare 函数进行排序
    std::sort(numbers.begin(), numbers.end(), compare);

    // 输出排序结果
    for (int num : numbers) {
        std::cout << num << " ";
    }
}
```

```
// 自定义比较函数
bool compare(int a, int b) {
    return a > b; // 降序排序
}

int main() {
    std::vector<int> numbers{ 3, 1, 4, 1, 5, 9, 2, 6 };

    // 使用 compare 函数进行排序
    std::sort(numbers.begin(), numbers.end(), compare);

    // 输出排序结果
    for (int num : numbers) {
        std::cout << num << " ";
    }
}
```

1. **代码分散**：比较函数只用于排序，却要单独定义，使代码不够紧凑。
2. **作用域污染**：函数名会暴露在全局作用域（或所在命名空间），可能引起命名冲突。
3. **阅读不便**：排序逻辑与调用分离，降低了可读性。

Lambda 表达式的优势

Lambda 允许我们将比较逻辑直接写在 `std::sort` 的调用中，无需额外定义函数：

```
std::sort(numbers.begin(), numbers.end(), [](int a, int b) {  
    return a > b;  
});
```

其中的 `[](int a, int b) { return a > b; }` 就是 Lambda 表达式。

6.11.2 Lambda 表达式的基本形式和语法

```
[capture-list] (parameter-list) -> return-type {  
    function-body  
};
```

- [capture-list]：捕获外部变量的方式，可以按值或按引用捕获。
- (parameter-list)：参数列表，类似于函数参数。
- -> return-type：返回类型，可省略（如果 Lambda 内部只有一条 return 语句，编译器会自动推导返回类型）。
- { function-body }：函数体，包含要执行的代码。

最简单的 Lambda：定义一个什么都不做的 Lambda：

```
[](){};           // 定义但不执行
```

这里，`[](){}` 是 Lambda 表达式：

- `[]`：空捕获列表，表示不访问外部变量。
- `()`：无参数。
- `{}`：Lambda 体，执行 `std::cout << "Hello, Lambda!" << std::endl;`。

如果需要定义并立即执行，可以这样：

```
[](){}();           // 定义并立即执行（无任何输出）
```

这里，`[](){}()` 是 Lambda 表达式：

- `[]`：空捕获列表，表示不访问外部变量。
- `()`：无参数。
- `{}`：Lambda 体，执行 `std::cout << "Hello, Lambda!" << std::endl;`。
- `()`：表示立即调用该 Lambda。

下面是一个更常见的例子：定义并立即执行一个打印信息的 Lambda：

```
[ ] { std::cout << "Hello, Lambda!\n"; }();    // 省略空参数列表，立即调用
```

这个 Lambda 没有参数，函数体包含一条输出语句。

6.11.3 Lambda 表达式的赋值

- Lambda 表达式本质上是一个**匿名函数**，但可以将它赋值给一个变量，从而在后续代码中多次调用该函数对象，无需重复编写 Lambda 代码。例如：

```
int main() {  
    auto f = []() {  
        std::cout << "Hello, Lambda!" << std::endl;  
    };  
    f();    // 调用  
    f();    // 再次调用  
}
```

6.11.4 Lambda的参数和返回值

Lambda 可以像普通函数一样接收参数和返回值。例如，一个求两数之和的 Lambda：

```
auto add = [](int a, int b) { return a + b; };  
std::cout << add(3, 4);    // 输出7
```

- 如果 Lambda 只有一条 return 语句，编译器可以自动推导返回类型。但如果函数体中有多个返回语句，或者需要明确指定返回类型，可以使用**尾置返回类型**——即在参数列表后面用 -> 加上返回类型。例如：

```
auto divide = [](int a, int b) -> double {  
    if (b == 0) return 0;  
    return static_cast<double>(a) / b;  
};
```

这里 -> double 显式指定了返回类型为 double 。如果不指定，编译器可能无法推导出正确的类型（例如两条返回语句的类型不一致时会导致编译错误）。

6.11.4 捕获外部变量

Lambda 可以访问其所在作用域中的局部变量，这称为**捕获**。捕获方式分为按值和按引用。

按值捕获：

将变量复制到 Lambda 内部，副本在 Lambda 定义时确定，后续修改外部变量不影响内部副本。

```
int x = 10;  
auto f = [x] { std::cout << x; };  
x = 20;  
f();    // 输出10
```

解释：

- `[x]` 按值捕获 `x`，Lambda 内部得到的是 `x` 的副本。
- 修改外部 `x` 不影响 Lambda 内部的 `x`。

按引用捕获

捕获变量的引用，Lambda 内部使用该引用操作原始变量。

```
int x = 10;  
auto f = [&x] { x++; };  
f();  
std::cout << x;    // 输出11
```

解释：

- `[&x]` 按引用捕获 `x`，Lambda 内部直接操作外部变量 `x`。

混合捕获

可以同时指定部分变量按值、部分按引用。

```
int a = 1, b = 2;  
auto f = [a, &b] { /* a按值, b按引用 */ };
```

隐式捕获

[=] 表示按值捕获所有用到的外部变量， [&] 表示按引用捕获所有用到的外部变量。

```
int x = 10, y = 20;  
auto f1 = [=] { return x + y; };    // 按值捕获x和y  
auto f2 = [&] { x++; y++; };        // 按引用捕获
```

显式捕获列表的常见形式：

[=] 表示按值捕获所有用到的外部变量， [&] 表示按引用捕获所有用到的外部变量。

- [x]：按值捕获 x
- [&x]：按引用捕获 x
- [=, &y]：按值捕获所有变量， y 按引用捕获
- [&, x]：按引用捕获所有变量， x 按值捕获

6.11.5 可变 Lambda (mutable)

默认情况下，按值捕获的变量在 Lambda 内部是只读的（相当于 const 副本）。尝试修改它会导致编译错误。

```
int x = 10;

auto lambda = [x]() {
    // x = 20;    // ✖ 编译错误: x 是只读的
    std::cout << "Inside Lambda, x = " << x << std::endl;
};

lambda();
```

6.11.5 可变 Lambda (mutable)

- 如果确实需要修改按值捕获的副本（而不影响外部变量），可以在 Lambda 参数列表后加上 mutable 关键字。此时 Lambda 内部可以修改副本，但外部变量仍保持不变。

```
int x = 10;

auto lambda = [x]() mutable {
    x++;    // ✅ 可以修改副本
    std::cout << "Inside Lambda, x = " << x << std::endl;
};

lambda();    // 输出: Inside Lambda, x = 11
std::cout << "Outside Lambda, x = " << x << std::endl;    // 输出: 10
```

6.11.6 初始化捕获 (init capture, C++14)

C++14 允许在捕获列表中直接定义并初始化变量，这些变量只属于 Lambda 内部，不会污染外部作用域。语法为 **[变量名 = 初始化表达式]**。

```
auto counter = [count = 0]() mutable {  
    return ++count;  
};  
std::cout << counter(); // 1  
std::cout << counter(); // 2
```

初始化捕获特别适合在 Lambda 中引入临时状态，而不必在外部定义变量。

6.11.7 Lambda 表达式的应用场景

1. STL 算法

标准库中的算法（如 `std::sort`、`std::find_if`、`std::for_each` 等）通常需要传入一个可调用对象（函数、函数指针或函数对象）来定制行为。

```
std::vector<int> numbers{1, 2, 3, 4, 5};  
std::for_each(numbers.begin(), numbers.end(), [](int& n) { n *= 2; });  
for (int n : numbers) std::cout << n << ' ';    // 输出 2 4 6 8 10
```

2. 回调函数

在事件驱动编程（如 GUI 程序、异步 I/O）中，回调函数用于响应特定事件。传统做法需要单独定义一个命名函数，如果回调逻辑简单且只在某处使用，使用 Lambda 可以省去命名麻烦，并将回调代码与事件注册放在一起，提高可读性。

示例：模拟按钮点击事件

```
button.onClick([]() { std::cout << "Button clicked!\n"; });
```

3. 临时函数逻辑

有时我们需要一段简单的逻辑，仅在一个地方使用（例如作为某个函数的参数，或用于初始化某个变量），但又不希望为了它专门定义一个命名函数污染作用域。Lambda 恰好提供了这种“即用即写”的能力，使代码更紧凑。

```
#include <algorithm>

int main() {
    std::vector<int> numbers{1, 2, 3, 4, 5};
    std::vector<int> squares;
    std::transform(numbers.begin(), numbers.end(),
                   std::back_inserter(squares),
                   [](int n) { return n * n; });
    for (int n : squares) std::cout << n << ' ';    // 输出 1 4 9 16 25
}
```

4. 并行计算与多线程

- C++11 开始提供了线程库（<thread>）和异步任务（<future>），Lambda 可以直接作为线程函数或任务函数传递，使并发代码更加简洁。

```
#include <iostream>
#include <future>

int main() {
    auto future = std::async(std::launch::async, []()
        std::cout << "Parallel task running...\n";
    });
    future.get(); // 等待任务完成
}
```

5. 事件监听与处理

在游戏开发、GUI 编程或自定义事件系统中，事件处理逻辑常常用 Lambda 表示。这可以将事件处理的代码直接写在事件订阅处，便于理解和维护。

示例：模拟事件订阅

```
eventSystem.subscribe("gameStart", [](const Event& e) {  
    std::cout << "Game started: " << e.message() << '\n';  
});
```

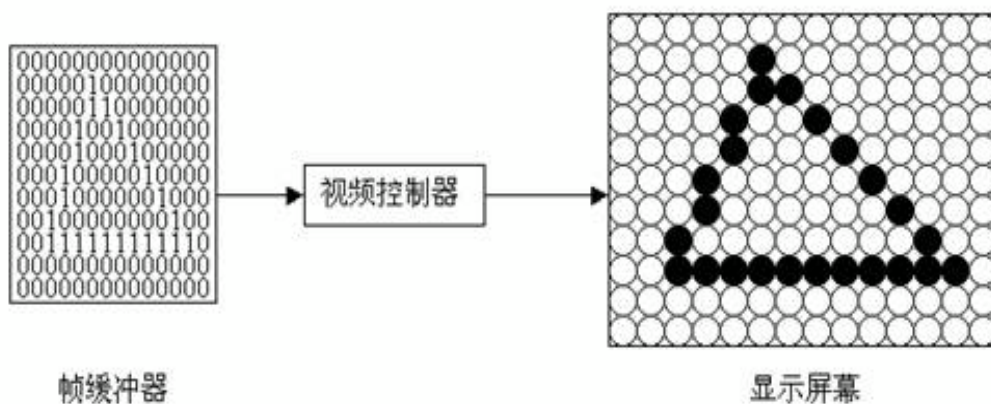
6.11.8 小结

- Lambda 是 C++11 引入的匿名函数，用于定义一次性使用的简短逻辑，尤其在 STL 算法和回调中非常实用。
- 基本语法：[捕获列表](参数列表) -> 返回类型 { 函数体 }，其中返回类型可省略。
- 捕获方式：按值 [=]、按引用 [&]、显式混合 [x, &y] 等。
- mutable 允许修改按值捕获的副本，但外部变量不受影响。
- C++14 支持初始化捕获，可在 Lambda 内部创建局部变量。

实战：二维字符图形库ChGL

彩色显示器和图形库OpenGL

- 彩色显示器的屏幕实际是由很多像素构成的



- OpenGL等图形库通过提供许多API函数给程序员，程序员调用这些API函数完成各种复杂的图形绘制任务。

二维字符图形库ChGL

- 控制台程序只能显示字符
- 控制台模拟彩色显示器：字符模拟像素，不同字符模拟像素颜色

`using color = char; //不同字符模拟像素颜色`

```
color* framebuffer{ nullptr };           //帧缓冲器
int framebuffer_width, framebuffer_height;
color clear_color{ ' ' };                //清屏颜色
```

frame_buffer: 存储画面图像

- frame_buffer是一个字符color (char) 类型的指针，指向动态分配的内存，用来表示帧缓冲器字符像素矩阵，

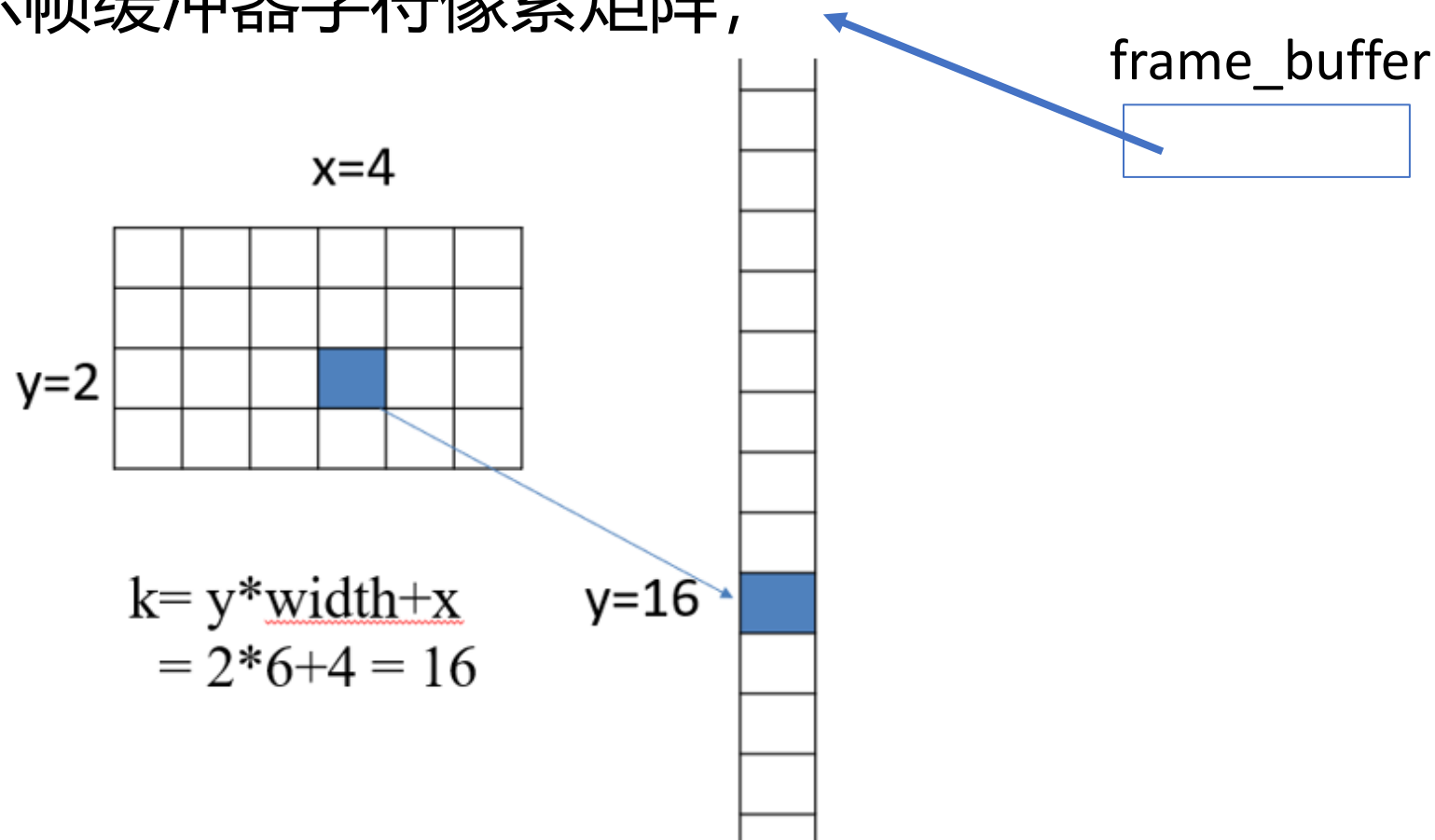


图 6-8 窗口像素坐标到一维存储下标的映射关系

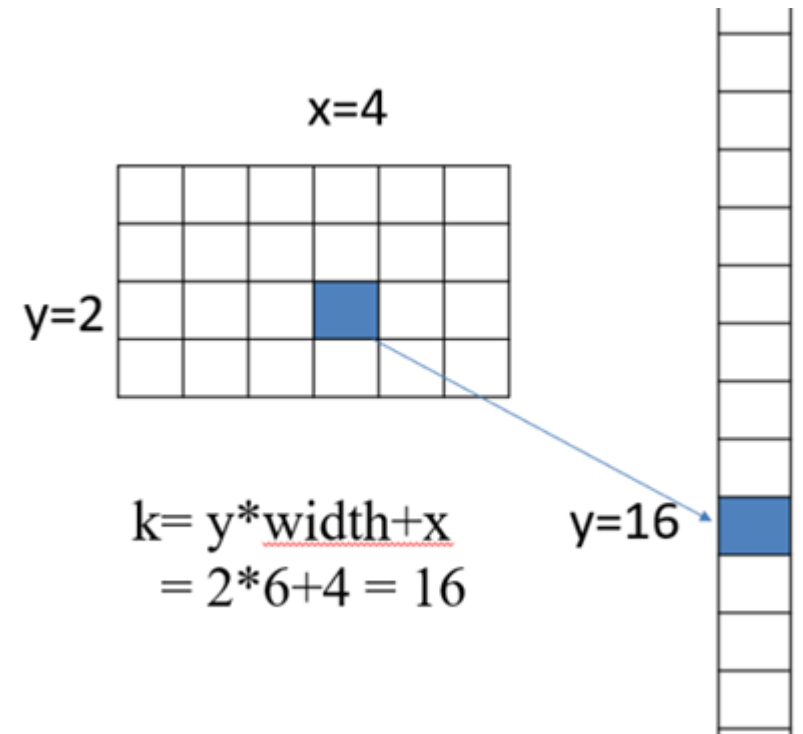
二维字符图形库ChGL

- API函数

```
bool initWindow(int width, int height); //初始化一个窗口, 返回bool值表示成功还是失败
void clearWindow(); //清空窗口内容
void destoryWindow(); //销毁窗口, 释放帧缓冲器占用的内存
void show(); //显示帧缓冲区的图像
void setPixel(const int x, const int y, color c = ' '); //设置像素的颜色
color getPixel(const int x, const int y); //设置像素的颜色
void set_clear_color(color c) { clear_color = c; }
color get_clear_color() { return clear_color; }
```

初始化窗口：分配framebuffer空间，清空颜色

```
bool initWindow(int width, int height) {  
    framebuffer = new color[width * height];  
    if (!framebuffer) return false;  
    framebuffer_width = width;  
    framebuffer_height = height;  
    clearWindow();  
    return true;  
}
```



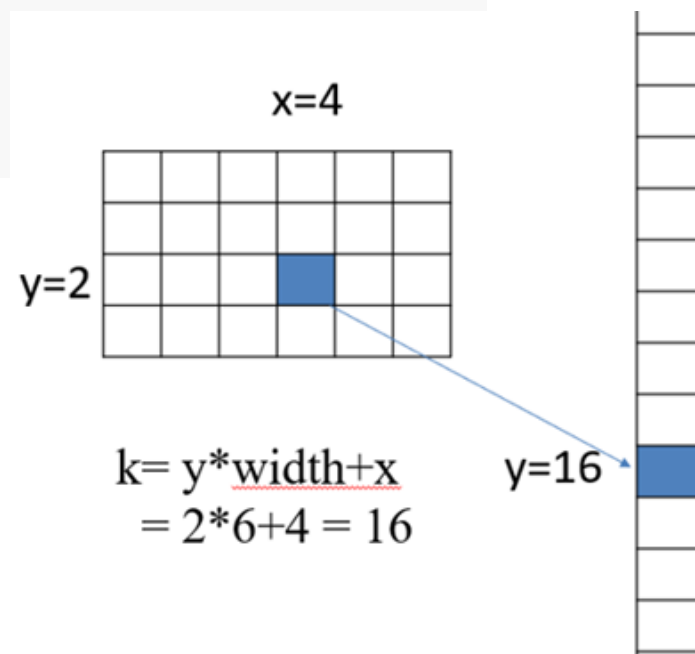
读、写framebuffer

```
void setPixel(const int x, const int y, color c) {  
    framebuffer[y * framebuffer_width + x] = c;  
}  
color getPixel(const int x, const int y) {  
    return framebuffer[y * framebuffer_width + x];  
}
```

清空窗口：将所有像素颜色设置为背景颜色

// 清空窗口：将帧缓冲器中每个像素设置为清屏颜色

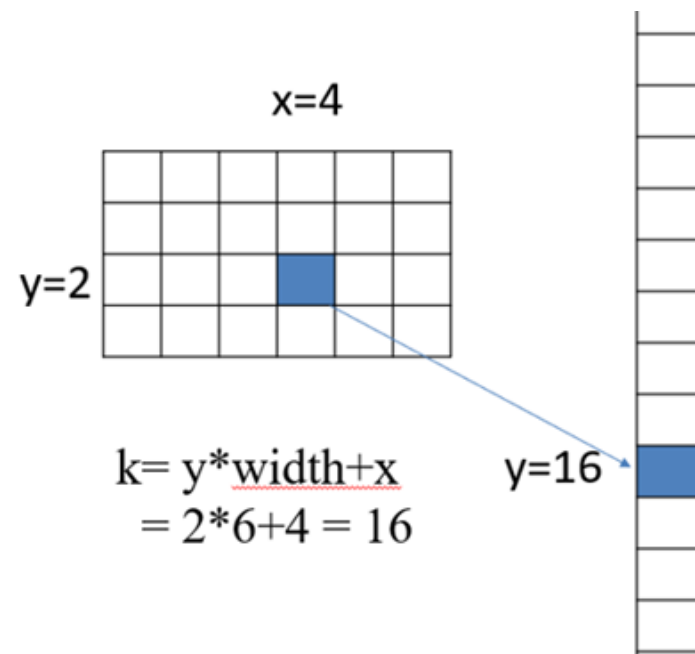
```
void clearwindow() {  
    auto framebuffer_size = framebuffer_width*framebuffer_height  
    for (auto k = 0; k != framebuffer_size; k++)  
        framebuffer[k] = clear_color;  
}
```



销毁窗口：释放帧缓冲器占用的内存

//销毁窗口，释放帧缓冲器占用的内存

```
void destoryWindow() {  
    delete[] framebuffer;  
    framebuffer = nullptr;  
}
```



显示帧缓冲区的图像

```
void show() {  
    for (int y = 0; y < framebuffer_height; y++) {  
        for (int x = 0; x < framebuffer_width; x++)  
            std::cout << framebuffer[y * framebuffer_width + x];  
        std::cout << '\n';  
    }  
}
```

测试一下

```
int main() {  
    if (!initWindow(25, 15)) {  
        return 1;  
    }  
    set_clear_color('-');  
    clearWindow();  
    int x{ 10 }, y{ 10 };  
    setPixel(x, y, '*');  
    setPixel(x - 1, y + 1, '*'); setPixel(x, y + 1, ' '); setPixel(x + 1, y + 1, '*');  
    setPixel(x - 2, y + 2, '*'); setPixel(x - 1, y + 2, ' '); setPixel(x, y + 2, '*'); setPixel(x + 1, y + 2, '*');  
    show();  
}
```

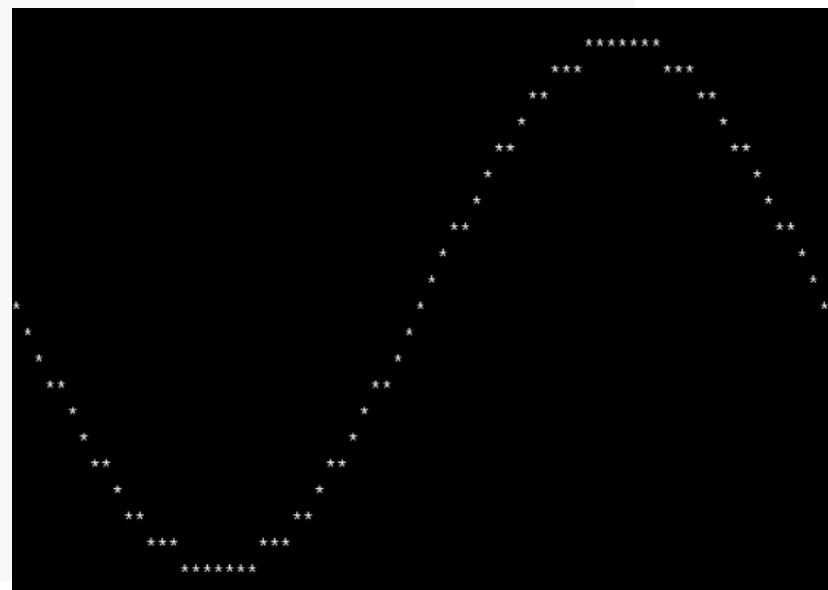


绘制一个正弦sin曲线。

```
#include <cmath>
const float PI{3.1415926};
void draw_sin_curve (int width,int height,int degree_step=5){

    for (int degree = 0 ; degree <= 360 ; degree = degree + degree_step ){
        auto x = floor( degree/360.*width) +1; //+1是为了防止x=0
        auto y = floor( (sin(degree*PI/180)+1) * height/2)+1;    //+1是为了防止y=0
        setPixel(x,y,'*');
    }
}

int main(){
    int width=50,height =40;
    if(!initwindow(width,height)){
        return 1;
    }
    draw_sin_curve(width,height-2);
    show();
}
```



优化屏幕刷新（屏幕绘制）函数

前面的屏幕刷新（屏幕绘制）`show()`函数存在一些明显的缺点：

1. 性能低下

- `std::cout` 逐个字符输出，会触发 **大量缓冲刷新**，导致性能下降。
- `std::cout << '\n'` 也会触发额外的输出缓冲管理，增加开销。

2. 缺少批量输出优化

- 逐字符输出比批量输出（如 `fwrite()` 或 `std::cout.write()`）慢很多。

3. 可能缺少换行符

- `framebuffer` 可能只是连续存储字符，没有 `\n`，导致必须在窗口每行结尾额外输出换行符 `\n`。

1. 逐步优化方案

使用 `putchar()` 替代 `std::cout`:

```
void show() {  
    for (int y = 0; y < framebuffer_height; y++) {  
        for (int x = 0; x < framebuffer_width; x++) {  
            putchar(framebuffer[y * framebuffer_width + x]);  
        }  
        putchar('\n'); // 手动换行  
    }  
}
```

改进点:  减少 `std::cout` 调用, `putchar()` 直接调用 C 标准库, 比 `std::cout` 更高效。

 仍然逐字符输出, 优化不彻底。

第二步：使用 `fwrite()` 批量输出

优化代码：

```
void show() {  
    for (int y = 0; y < framebuffer_height; y++) {  
        fwrite(&framebuffer[y * framebuffer_width], 1, framebuffer_width, stdout);  
        putchar('\n'); // 手动换行  
    }  
    fflush(stdout);  
}
```

改进点：  `fwrite()` 批量输出每一行，减少 `putchar()` 逐字符调用，提高性能。

 每次 `fwrite()` 后仍然需要 `putchar('\n')` 额外调用换行。

第三步：预先在 `framebuffer` 里存储换行符

核心思路：让 `framebuffer` 直接存储 `\n`，这样 `show()` 直接输出整个 `framebuffer`，避免手动添加换行符。

修改 `framebuffer` 结构：

```
framebuffer = new char[(framebuffer_width + 1) * framebuffer_height]; // 预留换行符
```

初始化 `framebuffer`（每行的最后加 `\n`）：

```
for (int y = 0; y < framebuffer_height; y++) {  
    framebuffer[y * (framebuffer_width + 1) + framebuffer_width] = '\n';  
}
```

优化 `show()` 代码：

```
void show() {  
    fwrite(framebuffer, 1, (framebuffer_width + 1) * framebuffer_height, stdout);  
    fflush(stdout);  
}
```

改进点：  彻底去掉 `putchar('\n')`，避免额外调用。

 直接 `fwrite()` 输出整个 `framebuffer`，大幅提高性能。

 内存占用稍微增加（每行额外存 `\n`，总共 `height` 个额外字节）。

另外，`setpixel`等函数也要做出相应的修改，考虑这个换行符：

// 初始化窗口，分配存储 `width * height` 个字符的内存，并用清屏颜色填充

```
bool initWindow(int width, int height) {  
    framebuffer_size = (width + 1) * height;  
    framebuffer = new color[framebuffer_size]; // 预留换行符  
    if (!framebuffer) return false;  
    framebuffer_width = width;  
    framebuffer_height = height;  
  
    for (int y = 0; y < framebuffer_height; y++) {  
        framebuffer[y * (framebuffer_width + 1) + framebuffer_width] = '\\n';  
    }  
    return true;  
}
```

```
void clearWindow() {  
    for (auto y = 0; y != framebuffer_height; y++)  
        for (auto x = 0; x != framebuffer_width; x++)  
            framebuffer[y * (framebuffer_width + 1) + x] = clear_color;  
}
```

// 根据坐标 (x, y) 设置对应像素的颜色

```
void setPixel(const int x, const int y, color c) {  
    framebuffer[y * (framebuffer_width + 1) + x] = c;  
}
```

// 根据坐标 (x, y) 获取对应像素的颜色

```
color getPixel(const int x, const int y) {  
    return framebuffer[y * (framebuffer_width + 1) + x];  
}
```

<https://github.com/hwdong-net/cplusplus17/blob/master/code/06.cpp>

控制台显示彩色

```
#include <iostream>
```

```
#ifdef _WIN32
```

```
#include <windows.h>
```

```
void setColor(int color) {
```

```
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), color);
```

```
}
```

```
#else
```

```
void setColor(const std::string& colorCode) {
```

```
    std::cout << "\033[" << colorCode << "m";
```

```
}
```

```
#endif
```

控制台显示彩色

```
int main() {  
    #ifdef _WIN32  
        setColor(4); // Windows: 红色  
        std::cout << "红色文字\n";  
        setColor(7); // 恢复默认  
    #else  
        setColor("31"); // Linux: 红色  
        std::cout << "红色文字\n";  
        setColor("0"); // 恢复默认  
    #endif  
    return 0;  
}
```

代码链接

<https://github.com/hwdong-net/cplusplus17/blob/master/code/06.cpp>

基于ChGL的Pong游戏

Youtube: hwdong

博客: <https://hwdong-net.github.io>

游戏的程序框架

- 游戏初始化
- 游戏的循环：
 - 处理用户输入
 - 更新游戏的数据
 - 绘制场景

```
int main(){↓  
    //1.初始化↓  
    init();↓  
    ↓  
    //2.游戏循环↓  
    while(running){↓  
        processInput();    //2.1 处理用户输入↓  
        update();           //2.2 更新游戏数据↓  
        renderScene();      //2.3 绘制场景↓  
    }    ↓  
    return 0;↓  
}←
```

用ChGL和函数重写Pong游戏

初始化游戏数据

- 游戏中的数据

```
int ball_x, ball_y, ball_vec_x{0}, ball_vec_y{ 0 }; //球位置和速度←
```

```
int paddle_w, paddle_h; //挡板的长宽←
```

```
int paddle1_x, paddle1_y, paddle1_vec{0}; //左挡板位置和速度←
```

```
int paddle2_x, paddle2_y, paddle2_vec{0}; //右挡板位置和速度←
```

```
int score1{ 0 }, score2{0}, score1_x, score1_y, score2_x, score2_y; //得分及得分的显示位置
```

```

bool init(const int window_width=100,const int window_height=40){
    if (!initWindow(window_width, window_height)) { // 初始化窗口
        return false;
    }
    ball_x = window_width / 2;
    ball_y = window_height / 2;

    paddle_w = 4; paddle_h = 10;
    paddle1_x = 0; paddle1_y = window_height / 2 - paddle_h / 2;
    paddle2_x = window_width - paddle_w; paddle2_y = paddle1_y;
    paddle1_vec = 3; paddle2_vec = 3;

    score1 = 0; score2 = 0;
    score1_x = paddle_w + 8; score1_y = 2;
    score2_x = window_width - 8 - paddle_w; score2_y=2;
    // auto pong_circle_r2{ 40 }; //禁区半径的平方
    srand((unsigned)time(0)); //生成随机数种子
    random_ball(window_width, window_height);
    return true;
}

```

//初始化球的位置和速度←

void random_ball(const int window_width, const int window_height) {←

 ball_x = window_width / 2; ball_y = window_height / 2;←

 ball_vec_x = rand() % 3 + 1; //生成一个随机整数表示球的横向速度←

 ball_vec_y = rand() % 3 + 1; //生成一个随机整数表示球的纵向速度←

 if (rand() % 2 == 1) ball_vec_x = -ball_vec_x; //速度可以使负数←

 if (rand() % 2 == 1) ball_vec_y = -ball_vec_y; //速度可以使负数←

}←

绘制背景

背景包括上下墙壁、左右沟渠和中间分割线，可以用一个函数绘制到画布上：

```
void draw_background() {  
    clearWindow();           //清空为背景颜色  
    int &window_width = framebuffer_width,&window_height = framebuffer_height;  
    auto right{ window_width - 1 }, middle{ window_width / 2 };  
    for (int y = 0; y < window_height; y++) {  
        setPixel(0, y, boundary_color);  
        setPixel(middle, y, boundary_color);  
        setPixel(right, y, boundary_color);  
    }  
    int bottom = window_height - 1;  
    for (int x = 0; x < window_width; x++) {  
        setPixel(x,0, wall_color);  
        setPixel(x,bottom,wall_color);  
    }  
}
```

绘制精灵(球和挡板)

//draw_sprites()绘制场景中的精灵：球、挡板和得分。↵

```
void draw_sprites() {    ↵
```

```
    //绘制球↵
```

```
    setPixel(ball_x, ball_y, ball_color);    ↵
```

```
    //绘制左、右挡板↵
```

```
    for (auto y = paddle1_y; y < paddle1_y + paddle_h; y++)↵
```

```
        for (auto x = paddle1_x; x < paddle1_x + paddle_w; x++)↵
```

```
            setPixel(x,y, paddle_color);    ↵
```

绘制精灵(球和挡板)

```
for (auto y = paddle2_y; y < paddle2_y + paddle_h; y++)↵
    for (auto x = paddle2_x; x < paddle2_x + paddle_w; x++)↵
        setPixel(x,y, paddle_color);↵
    ↵
//绘制分数： 分数是一个字符串↵
std::string s1 { std::to_string(score1) }, s2 { std::to_string(score2) };↵
for (auto i = 0; i < s1.size(); i++)↵
    setPixel(score1_x + i, score1_y, s1[i]);↵
for (auto i = 0; i < s2.size(); i++)↵
    setPixel(score2_x + i, score2_y, s2[i]);    ↵
}↵
```


绘制场景

```
void gotoxy(int x, int y) {  
    COORD coord = {x, y };  
    SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), coord);  
}  
  
void hideCursor() {  
    CONSOLE_CURSOR_INFO cursor_info = { 1, 0 };  
    SetConsoleCursorInfo(GetStdHandle(STD_OUTPUT_HANDLE), &cursor_info);  
}  
  
void render_scene() {  
    gotoxy(0, 0); //定位到(0,0)，相当于清空屏幕  
    hideCursor();  
    draw_background(); //在画布上绘制背景  
    draw_sprites(); //在画布上绘制精灵  
    show(); //在屏幕上显示画布内容（场景）  
}
```

测试一下

```
int main() {  
    if (!init()) {  
        std::cout << "初始化窗口失败！ \n";  
        return 1;   
    }  
    render_scene();  
    return 0;  
}
```

事件处理

```
int processInput() {  
    // 处理事件  
    char key;  
    if (_kbhit()) {  
        key = _getch();  
        if (key == 27) return -1;  
        else if ((key == 'w' || key == 'W') && paddle1_y > paddle1_vec)  
            paddle1_y -= paddle1_vec;  
        else if ((key == 's' || key == 'S') && paddle1_y + paddle1_vec + paddle_h < HEIGHT)  
            paddle1_y += paddle1_vec;  
        else if (key == 72 && paddle2_y > paddle2_vec)  
            paddle2_y -= paddle2_vec;  
        else if ((key == 80) && paddle2_y + paddle2_vec + paddle_h < HEIGHT)  
            paddle2_y += paddle2_vec;  
    }  
    return 0;  
}
```

更新游戏状态（数据）

更新球的位置，检测球与墙壁、挡板是否发生碰撞。↵

```
void update() {↵  
    // 2. 更新数据 ↵  
    ball_x += ball_vec_x;↵  
    ball_y += ball_vec_y;↵  
    if (ball_y < 0 || ball_y >= HEIGHT) {↵  
        ball_vec_y = -ball_vec_y; ↵  
        ball_y += ball_vec_y;↵  
    }↵
```

更新游戏状态 (数据)

```
if (ball_x < paddle_w && ball_y >= paddle1_y && ball_y < paddle1_y + paddle_h) {  
    ball_vec_x = -ball_vec_x;  
    score1 += 1;  
}  
else if (ball_x > WIDTH - paddle_w && ball_y >= paddle2_y && ball_y < paddle2_y + paddle_h) {  
    ball_vec_x = -ball_vec_x;  
    score2 += 1;  
}  
bool is_out{ false };  
if (ball_x < 0) { score2 += 1; is_out = true; }  
else if (ball_x >= WIDTH) { score1 += 1; is_out = true; }  
if (is_out) {  
    random_ball();  
}  
}
```

主函数

```
int main() {  
    //1. 初始化数据  
    init();  
    //2. 游戏循环  
    while (true) {  
        if (processInput() < 0) break;  
        update();  
        render_scene();  
    }  
    return 0;  
}
```

完整代码

https://github.com/hwdong-net/cplusplus17/blob/master/code/6.9%20pong_fun.cpp

关注我

博客: hwdong-net.github.io

Youtube频道



hwdong

@hwdong · 5.01K subscribers · 558 videos

博客: <https://hwdong-net.github.io> >

youtube.com/c/4kRealSound and 4 more links